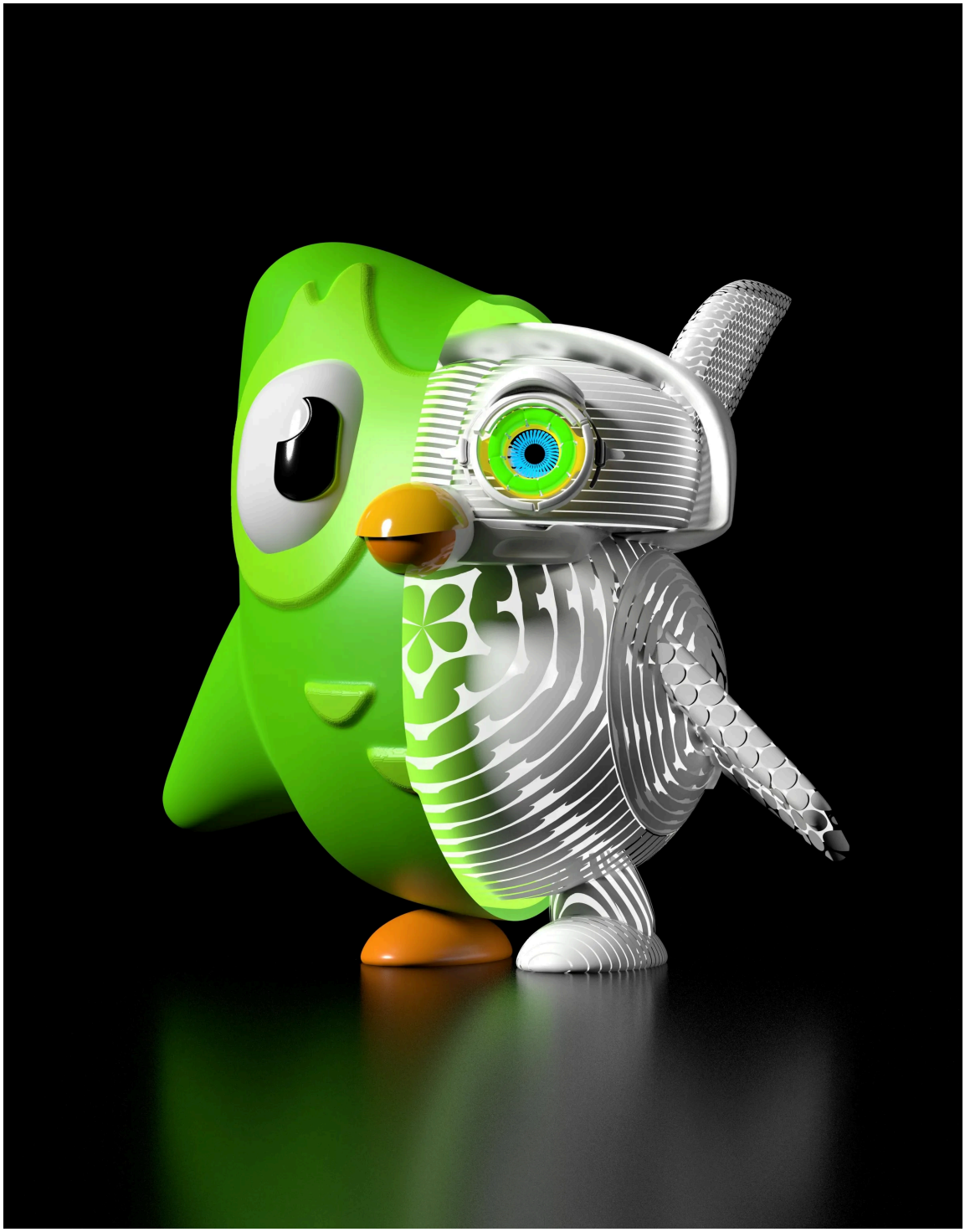
Entraîner un réseau de neurones artificiels (partie 2)

CSI 4506 - automne 2026

Marcel Turcotte
Version: nov. 1, 2026 17h45

Préambule

Message du jour



Comment l'IA de Duolingo apprend ce que vous devez apprendre, Bicknell et al. (2023).

L'article décrit l'évolution de Birdbrain, le système d'IA de Duolingo : un modèle de régression logistique mis à jour chaque nuit par traitement en lot a cédé la place à un système en temps réel utilisant un réseau de neurones récurrent à mémoire à long et à court terme (LSTM).

Objectifs d'apprentissage

- **Algorithme de rétropropagation** :
  - **Expliquer** les signaux d'erreur et la règle de la chaîne le long d'un chemin et aux embranchements.
  - **Relier** les dérivées scalaires aux implémentations Python et NumPy.
  - **Distinguer** le calcul des gradients de la mise à jour des paramètres.
- **Implémentation** :
  - **Expliquer** un perceptron multicouche à unités sigmoïdes qui conserve les valeurs du passage avant et utilise la rétropropagation par mini-lots.

La présentation précédente a développé un réseau de neurones à propagation avant pour en illustrer le fonctionnement. Un algorithme d'entraînement naïf, sans dérivées, a permis de distinguer le calcul effectué par le réseau de la méthode utilisée pour apprendre ses paramètres. Cette présentation porte sur la rétropropagation (*backpropagation*), un algorithme central de l'apprentissage profond.

Notation

Unités, couches et fonctions d'activation

Symbole	Signification
$z^{(l)}$	Préactivations : sommes pondérées et biais de la couche l
$a^{(l)}$	Activations : sorties de la couche, notées o pour les unités dans la présentation 11
φ	Fonction d'activation générale, appliquée élément par élément
σ	Fonction d'activation sigmoïde logistique
$W^{(l)}$, $b^{(l)}$	Poids et biais de la couche

Symbole	Signification
f_l	Transformation complète d'une couche : calcul affine suivi de l'activation

Le symbole φ désigne toujours une fonction d'activation, tandis que f_l désigne une couche entière, y compris ses poids et ses biais. La sigmoïde est notée σ. Les schémas révisés utilisent f_l plutôt que ϕ_l pour les transformations complètes des couches. La distinction entre une activation scalaire et une couche complète ne dépend donc pas de la reconnaissance des deux variantes visuellement proches ϕ et φ.

Les caractéristiques d'entrée des schémas sont notées x_i plutôt que $x^{(i)}$. L'indice inférieur i désigne une caractéristique, tandis que les indices supérieurs entre parenthèses, comme dans $a^{(l)}$, $z^{(l)}$ et $W^{(l)}$, désignent une couche. Cette convention distingue les indices des caractéristiques de ceux des couches et assure la cohérence entre les schémas et les équations.

L'implémentation utilise `a` pour les activations. Ces quantités correspondent aux sorties des unités notées o dans la présentation 11. Les minuscules a et z décrivent un exemple, tandis que les majuscules A et Z décrivent un lot d'exemples stockés en lignes. Les activations d'entrée sont $a^{(0)} = x$ pour un exemple et $A^{(0)} = X$ pour un lot. L'indice de couche s'écrit l, tandis que la perte d'un exemple s'écrit ℓ.

Les couches mathématiques sont numérotées à partir de 1, tandis que les listes Python sont indexées à partir de 0 : `self.W[0]` et `self.b[0]` correspondent à $W^{(1)}$ et $b^{(1)}$. Dans un poids $w_{i,j}^{(l)}$, i désigne une unité de la couche précédente et j une unité de la couche courante.

Probabilités, étiquettes et perte

Pour la classification binaire, comme dans les présentations 6 et 12 :

- $\hat{p}$ est la probabilité prédite de la classe 1.
- $\hat{y} = \mathbf{1}[\hat{p} \geq 0.5]$ est l'étiquette de classe prédite.
- ℓ est la perte d'**un exemple** et J est la **perte moyenne**.

$$\ell(y, \hat{p}) = -\left[y \log \hat{p} + (1 - y) \log(1 - \hat{p})\right]$$

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(y_i, \hat{p}_i).$$

La sortie sigmoïde du réseau est une probabilité. L'application d'un seuil produit une étiquette de classe, utile pour calculer l'exactitude, mais élimine de l'information nécessaire au calcul de la perte. C'est pourquoi la présentation 12 utilisait `bce_loss(y, self.forward(X))` plutôt que `bce_loss(y, self.predict(X))`.

La rétropropagation différencie la perte à travers les opérations qui produisent la probabilité. L'application du seuil reste en dehors de ce calcul. Dans l'exemple scalaire, ℓ désigne la perte de l'unique exemple traité. La moyenne sur les exemples sera introduite lors du retour aux lots.

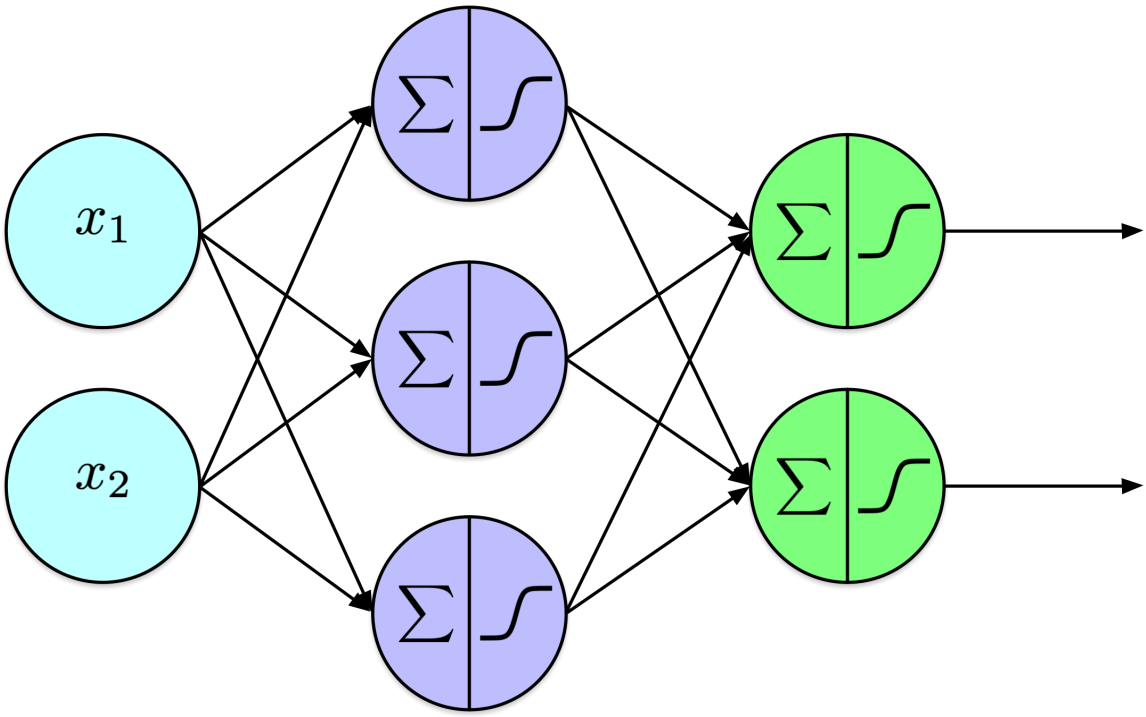
Notation : réseau à deux couches

Une couche cachée et une couche de sortie :

$$a^{(2)} = f_2(f_1(x))$$

Chaque couche calcule

$$f_l(a) = \varphi(aW^{(l)} + b^{(l)}).$$



Cette écriture suit la convention NumPy de la présentation 12 : `a @ W + b`. Ici, x et chaque vecteur d'activations sont des vecteurs lignes. La matrice des poids a la forme `(units_in, units_out)`. Les nœuds d'entrée transmettent les caractéristiques sans les modifier et ne comptent pas parmi les couches paramétrées.

Le schéma permet plusieurs unités de sortie : $a^{(2)}$ désigne donc le vecteur de sortie complet. Nos exemples de classification binaire utilisent une seule unité de sortie

sigmoïde, dont l'activation est la probabilité prédite $\hat{p}$. Des couches différentes peuvent utiliser des fonctions d'activation différentes. Le symbole φ représente l'activation choisie pour la couche considérée.

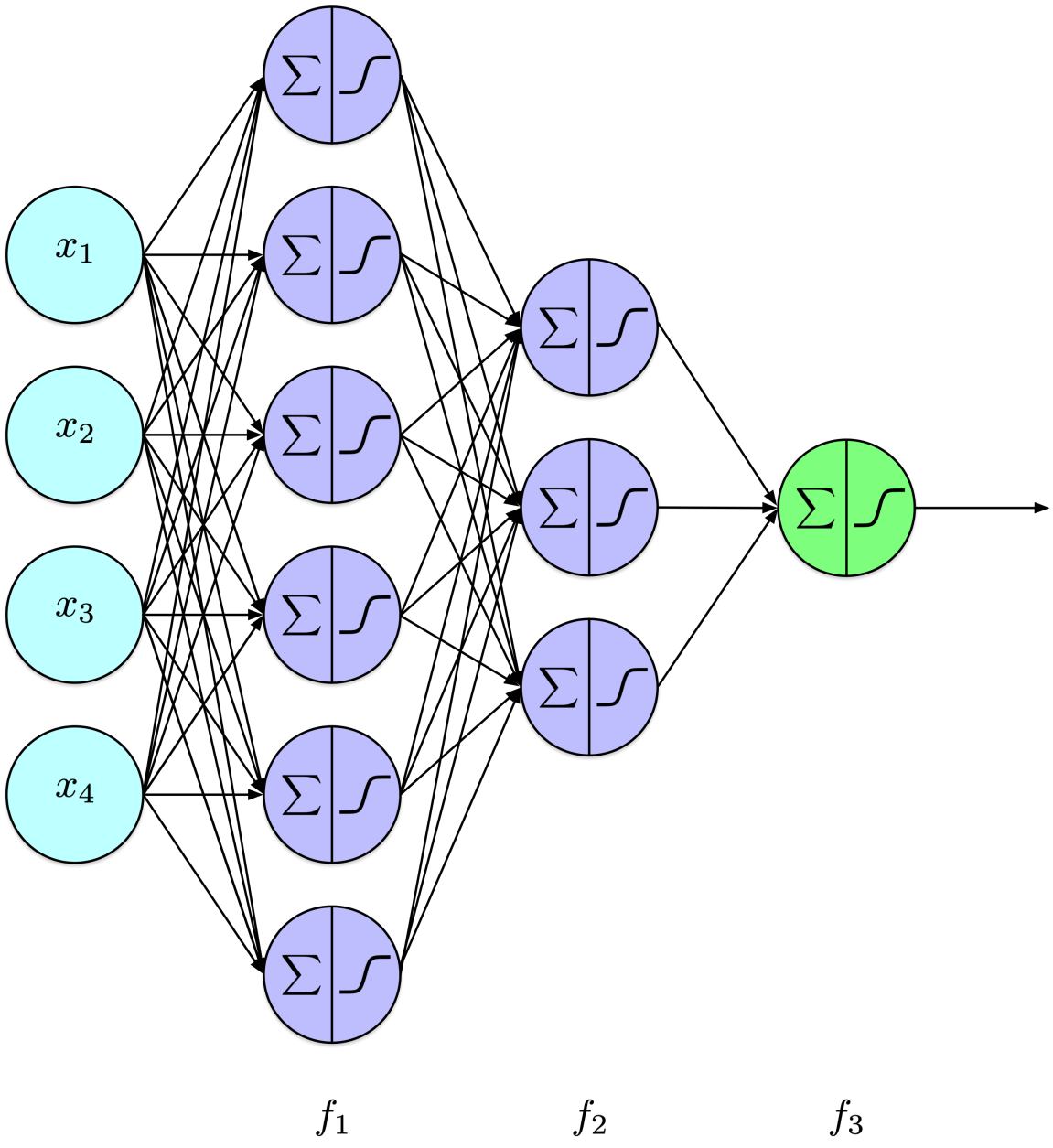
Notation : réseau à trois couches

Deux couches cachées et une couche de sortie :

$$a^{(3)} = f_3(f_2(f_1(x)))$$

où

$$f_l(a) = \varphi(aW^{(l)} + b^{(l)}).$$



Notation : réseau à L couches

$$a^{(L)} = f_L(\dots f_2(f_1(x))\dots).$$

Pour $l = 1, \dots, L$:

$$z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)}, \qquad a^{(l)} = \varphi(z^{(l)}).$$

La sortie d'une couche devient l'entrée de la suivante.

Seule la première couche paramétrée reçoit les caractéristiques d'entrée originales. Les couches suivantes traitent les activations de la couche précédente. Chaque couche répète le même schéma de calcul avec ses propres paramètres.

Pour un lot, remplacez x, a et z par X, A et Z. Chaque ligne décrit alors un exemple, et NumPy diffuse le vecteur de biais sur les lignes. La sortie est $A^{(L)} = f_L(\dots f_1(X)\dots)$.

Ajouter des couches augmente les calculs et le nombre de valeurs intermédiaires à conserver. La structure régulière permet d'étendre les mêmes procédures de propagation avant et arrière à des réseaux plus profonds, mais le coût reste dépendant de la profondeur.

Rétropropagation



Attribution : xkcd.com/1838

Rétropropagation

Learning representations by back-propagating errors

David E. Rumelhart, Geoffrey E. Hinton & Ronald J. Williams

Nous décrivons une nouvelle procédure d'apprentissage, la rétropropagation, pour des réseaux d'unités semblables à des neurones. La procédure **ajuste les poids de façon répétée** dans les connexions du réseau afin de **minimiser une mesure de la différence entre le vecteur de sortie réel du réseau et le vecteur de sortie souhaité**. Ces

ajustements amènent les unités internes « **cachées** », qui ne font partie ni de l'entrée ni de la sortie, à **représenter des caractéristiques importantes du domaine de la tâche**. Les interactions entre ces unités capturent les régularités de la tâche. La **capacité de créer de nouvelles caractéristiques utiles** distingue la rétropropagation des méthodes antérieures plus simples, comme la procédure de convergence du perceptron.

Rumelhart et al. (1986)

Ce résumé est traduit de l'article influent publié en 1986 par Rumelhart, Hinton et Williams, qui a contribué à populariser la rétropropagation pour apprendre des représentations internes. Des travaux antérieurs sur la différentiation automatique en mode inverse précèdent cet article (Baydin et al. 2017). Le résumé relie une fonction de perte, les mises à jour des paramètres fondées sur le gradient et les couches cachées qui apprennent des caractéristiques utiles pour la tâche.

Nature est une revue prestigieuse qui publie occasionnellement des travaux liés à l'informatique.

Au moment de cette publication, Hinton était affilié à l'Université Carnegie Mellon. Rappelons qu'il a reçu le prix Nobel de physique en 2024 pour ses contributions au développement de méthodes fondamentales de l'apprentissage automatique moderne.

Le résumé souligne l'intérêt des couches cachées dans les réseaux de neurones. Les premières couches cachées apprennent des représentations simples à partir des données d'entrée, tandis que les suivantes identifient des associations entre ces représentations. Chaque couche exploite les représentations des couches précédentes jusqu'à la sortie finale du réseau.

Représentation et apprentissage

- Un **perceptron à une seule couche** ne peut séparer que des classes **linéairement séparables**.
- Un **réseau multicouche peut représenter XOR**, comme l'a montré la présentation 11.
- Apprendre les paramètres de ses couches cachées exige de résoudre le **problème d'attribution du crédit** (*credit assignment problem*) : comment chaque paramètre influence-t-il la perte finale?

Représenter une fonction et apprendre des paramètres convenables sont deux questions différentes. Dans l'exemple XOR de la présentation 11, les poids étaient fournis manuellement. Pour une nouvelle tâche, l'entraînement doit déterminer comment chaque paramètre d'une couche cachée influence la perte, même si aucune valeur cible n'est fournie pour les unités cachées. La rétropropagation calcule efficacement ces

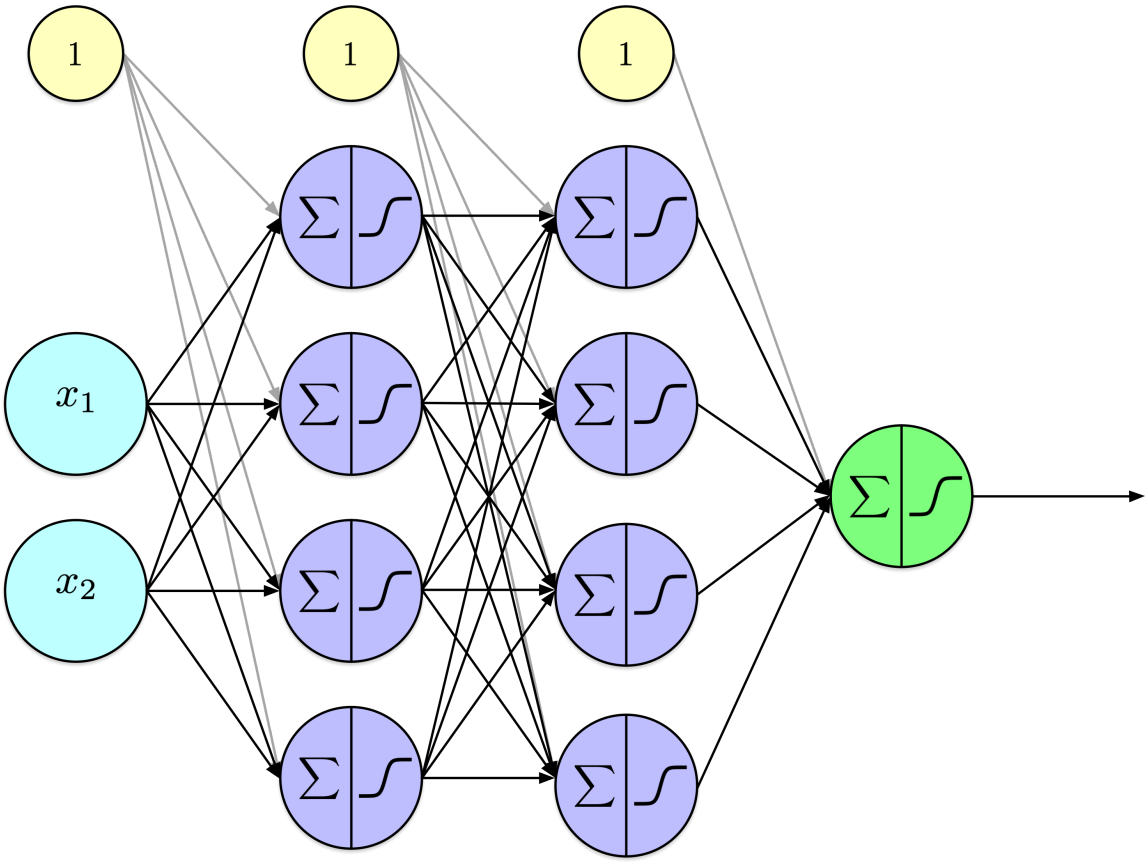
sensibilités. Les limites de modèles de perceptrons restreints ne doivent pas être interprétées comme des limites de tous les réseaux multicouches.

La rétropropagation dans l'article de 1986

- L'article utilisait un **objectif d'erreur quadratique** et des **unités sigmoïdes**.
- Une **propagation avant** calcule les activations, puis une **propagation arrière** calcule les gradients des paramètres.
- La **descente de gradient** utilise ces gradients pour mettre les paramètres à jour.
- Les unités cachées peuvent apprendre des **représentations internes utiles**.

Ces points décrivent l'article historique (Rumelhart et al. 1986). La rétropropagation n'est liée ni à l'erreur quadratique ni aux unités cachées sigmoïdes. Notre exemple de classification binaire utilise des unités sigmoïdes et l'entropie croisée binaire, dans la continuité des présentations 6 et 12. Les choix de perte et d'activation déterminent les dérivées locales, tandis que le calcul arrière suit les mêmes principes de la règle de la chaîne.

Idée conceptuelle



$$J(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$

Avec un réseau dont les paramètres ont été initialisés aléatoirement, nous pouvons produire des probabilités ($\hat{p}_i$), mais les paramètres n'ont pas encore été ajustés à la tâche.

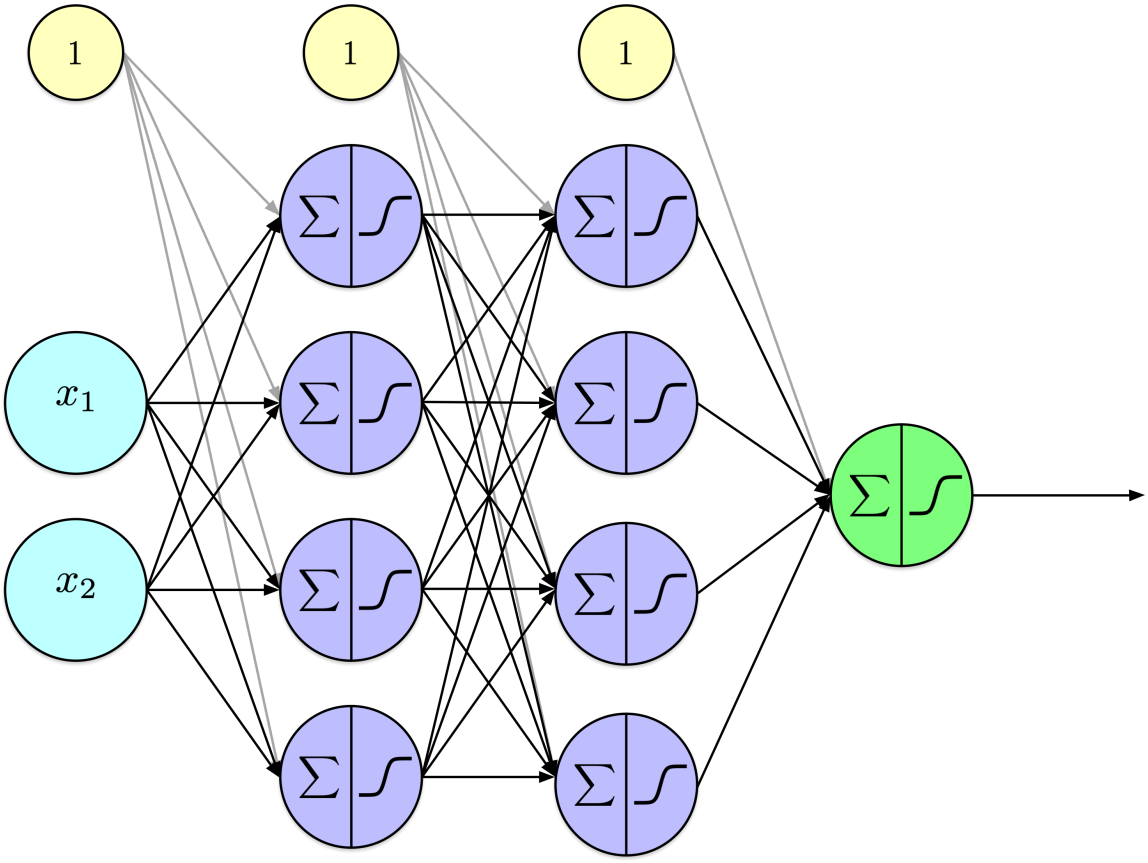
Dans un problème de classification binaire, à quel niveau d'exactitude peut-on s'attendre?

Un classificateur qui devine les étiquettes indépendamment avec des probabilités égales a une exactitude attendue de 50 %. C'est un point de comparaison utile pour une tâche binaire équilibrée, mais un réseau particulier initialisé aléatoirement n'atteint pas nécessairement cette exactitude.

Pour évaluer les prédictions du modèle, nous utilisons ici l'entropie croisée binaire (*binary cross-entropy*, BCE), aussi appelée log-vraisemblance négative. La fonction de perte calcule la moyenne des pertes sur tous les exemples d'entraînement, comme dans la présentation 12. Le terme à l'intérieur de la somme calcule la perte d'un exemple.

Dans ce réseau binaire, l'unique activation de sortie est $\hat{p}_i = f_L(\ldots f_2(f_1(x_i))\ldots)$. Ici, x_i désigne l'exemple i du jeu de données. Lorsqu'il est question des caractéristiques d'un exemple, x_j désigne la caractéristique j. La transformation complète d'une couche est $f_l(a) = \varphi(aW^{(l)} + b^{(l)})$, où φ est l'activation choisie pour cette couche.

Idée conceptuelle (suite)



$\frac{\partial}{\partial \theta_k} J(\theta)$, pour tout k, où les θ_k sont les `W[l][i,j]` et `b[l][j]`.

Dans cette notation, l'indice `l` désigne les couches de traitement du modèle. L'indice `i` désigne une unité de la couche précédente, notée `l-1`, tandis que `j` désigne une unité de la couche courante `l`.

Prenez un moment pour interpréter ces dérivées partielles. Chacune quantifie le taux de variation local de la perte par rapport à un seul paramètre, tous les autres paramètres restant fixes. Son signe indique si une petite augmentation du paramètre augmente ou diminue la perte. Sa valeur absolue mesure la sensibilité de la perte à ce changement.

La descente de gradient soustrait un multiple de chaque dérivée. Une dérivée positive entraîne une diminution du paramètre, et une dérivée négative entraîne une augmentation. Ces indications sont locales : le taux d'apprentissage reste donc important. Chaque dérivée partielle est une composante du vecteur gradient $\nabla_\theta J$.

L'indice mathématique l commence à 1, tandis que l'indice de couche `l` en Python commence à 0. Ainsi, `W[l][i, j]` correspond à l'entrée d'indices mathématiques $i+1, j+1$ dans $W^{(l+1)}$.

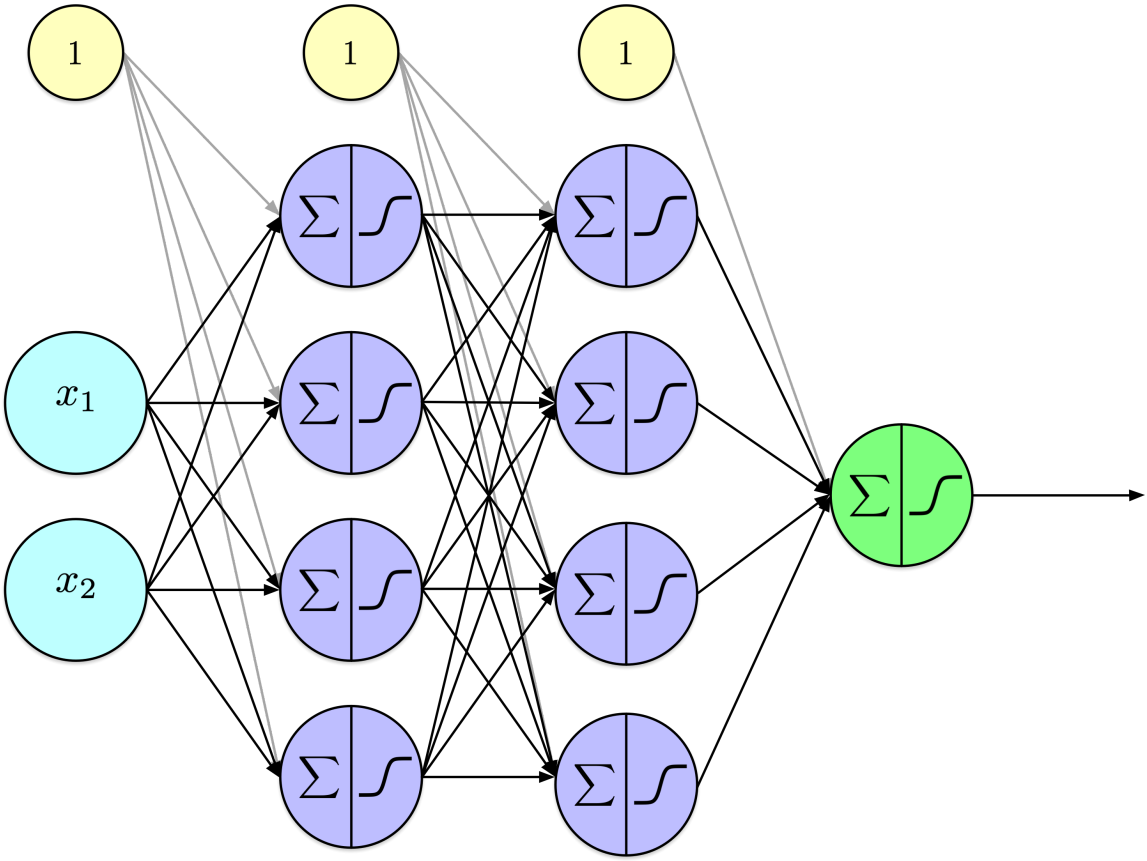
Combien de paramètres le modèle contient-il?

Le modèle contient trois matrices de poids, de dimensions 2×4, 4×4 et 4×1. Elles contiennent respectivement 8, 16 et 4 poids, pour un total de 28 paramètres de poids.

Il contient aussi trois tableaux de biais, avec respectivement 4, 4 et 1 biais, soit 9 paramètres de biais.

Le modèle compte donc 37 paramètres, ce qui est relativement peu par rapport à des modèles plus complexes. Par exemple, le modèle Fashion-MNIST de la présentation 11 comptait 266 610 paramètres. En apprentissage automatique contemporain, les modèles peuvent contenir des millions, voire des milliards de paramètres.

Idée conceptuelle (suite)



Pour un nombre fixé d'époques : $\theta = \theta - \alpha \nabla_\theta J(\theta)$

Les algorithmes d'entraînement utilisent aussi d'autres critères d'arrêt. Par exemple, l'entraînement peut s'arrêter lorsque la perte de validation ne s'est pas améliorée depuis un nombre prédéfini d'époques.

Les paramètres du modèle sont mis à jour simultanément, et ces mises à jour peuvent être calculées efficacement en parallèle.

Ce processus s'appelle la descente de gradient. Lorsque chaque mise à jour utilise le jeu de données complet, on parle de **descente de gradient par lot**. Lorsqu'elle utilise un seul exemple, il s'agit de **descente de gradient stochastique**. Enfin, lorsqu'elle utilise un petit sous-ensemble d'exemples, appelé mini-lot, on parle de **descente de gradient par mini-lots** (*mini-batch gradient descent*).

Il reste à déterminer les dérivées partielles de la perte par rapport à chacun des 37 paramètres du modèle.

Rétropropagation

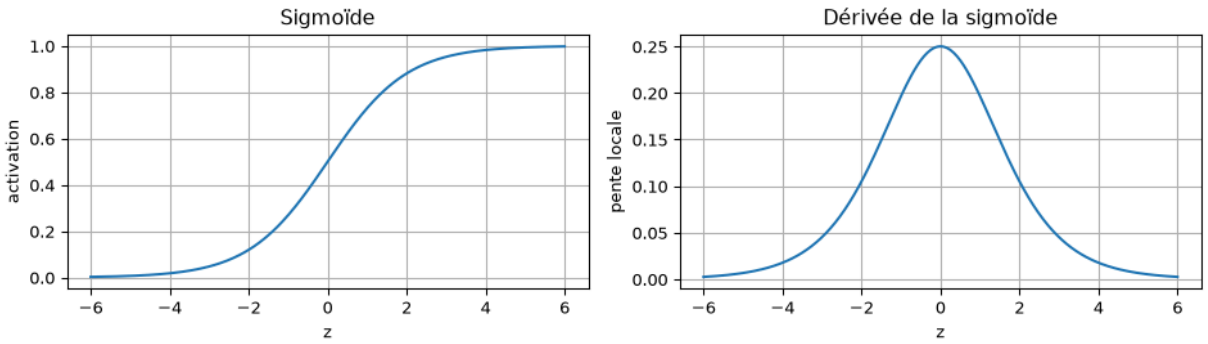
- La **rétropropagation** est un algorithme qui calcule **méthodiquement** les **dérivées partielles** de la fonction de perte d'un réseau de neurones par rapport à chaque paramètre de **poids** et de **biais**.
- La **rétropropagation** applique **récursivement** la **règle de la chaîne** pour calculer $\frac{\partial J}{\partial w_{i,j}^{(l)}}$ et $\frac{\partial J}{\partial b_j^{(l)}}$ pour chaque couche l, en réutilisant efficacement les quantités intermédiaires du passage avant.

La sigmoïde et sa dérivée

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)).$$

In [2]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit

z_values = np.linspace(-6, 6, 300)
a_values = expit(z_values)
fig, axes = plt.subplots(1, 2, figsize=(10, 3))
axes[0].plot(z_values, a_values)
axes[0].set(title="Sigmoïde", xlabel="z", ylabel="activation")
axes[1].plot(z_values, a_values * (1 - a_values))
axes[1].set(title="Dérivée de la sigmoïde", xlabel="z", ylabel="pente locale
for ax in axes:
    ax.grid(True)
plt.tight_layout()
plt.show()
```



Vous n'avez pas à calculer manuellement chaque dérivée locale. Une bibliothèque de calcul symbolique comme SymPy peut calculer ou vérifier ces expressions. Vous devez comprendre le rôle de la règle de la chaîne

combiner chaque dérivée locale avec la sensibilité de la perte reçue des opérations en aval.

Par exemple, ce calcul SymPy vérifie que la dérivée de la sigmoïde est $\sigma(z)(1-\sigma(z))$:

```
import sympy as sp
z = sp.symbols("z", real=True)
s = 1 / (1 + sp.exp(-z))
sp.simplify(sp.diff(s, z) - s * (1 - s))  # 0
```

Consultez le tutoriel de différentiation de SymPy pour la fonction `diff`. SymPy manipule des expressions symboliques. La différentiation automatique propage plutôt les valeurs des dérivées à travers les opérations exécutées par un programme.

La dérivée de la sigmoïde atteint son maximum de 0.25 à $z = 0$ et tend vers zéro lorsque z tend vers l'un ou l'autre infini. La dérivée est évaluée à la préactivation z. Si l'activation $a = \sigma(z)$ est déjà connue, sa dérivée locale se calcule par $a(1-a)$. La présentation 14 examine l'effet de la multiplication répétée de petits facteurs sur la propagation des gradients dans les réseaux plus profonds.

Règle de la chaîne

Soit

$$h(x) = f(g(x))$$

en notation de **Lagrange**, nous avons

$$h'(x) = f'(g(x))g'(x)$$

ou, de façon équivalente, en notation de **Leibniz**

$$\frac{dv}{dx} = \frac{dv}{du} \cdot \frac{du}{dx}, \qquad u = g(x), \quad v = f(u).$$

La **règle de la chaîne** permet de calculer la dérivée d'une **fonction composée**. Si une fonction h est définie comme la composition de deux fonctions différentiables f et g, elle fournit une méthode pour calculer h'.

Si l'on connaît le taux de variation instantané de v par rapport à u et celui de u par rapport à x, on obtient le taux de variation de v par rapport à x en multipliant ces deux taux. Les variables intermédiaires u et v évitent de réutiliser y, qui désigne une cible dans nos exemples de classification.

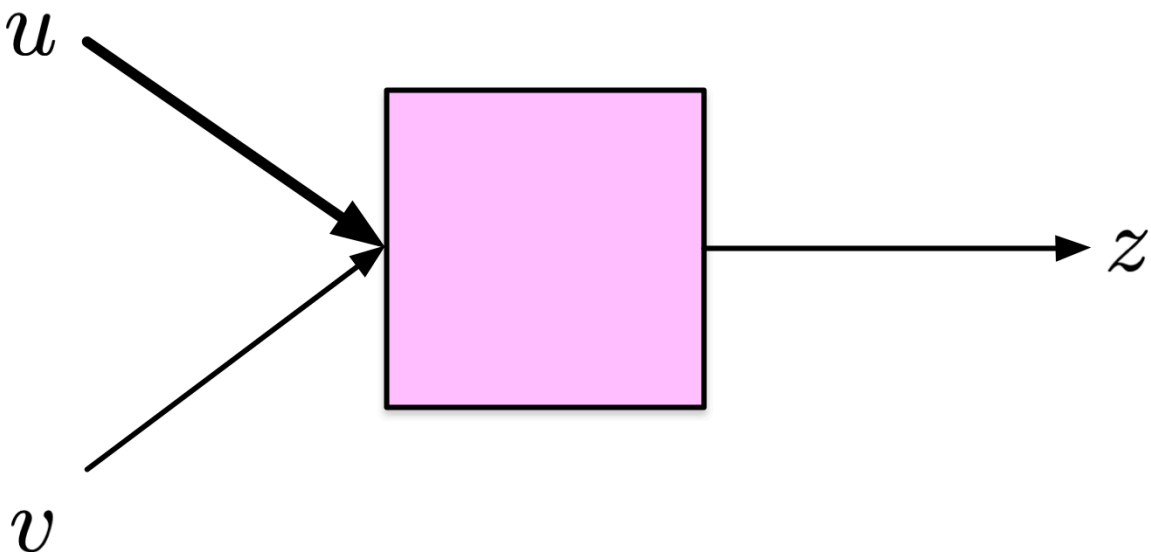
La même composition apparaît dans la notation des couches $a^{(L)} = f_L(\dots f_2(f_1(x))\dots)$. La rétropropagation applique la règle de la chaîne aux opérations de ces couches et au calcul de la perte.

Quelques vérifications utiles lors du calcul des dérivées :

- Distinguez la valeur d'une activation de sa pente. La dérivée locale d'une activation n'est qu'un facteur du gradient d'un paramètre.
- Évaluez les dérivées locales aux valeurs du passage avant et gardez les paramètres fixes pendant le passage arrière.
- Multipliez le long d'un chemin et additionnez les contributions aux embranchements. Appliquez une seule fois le facteur de moyenne sur les exemples.
- Un gradient nul peut provenir d'une saturation ou d'une annulation. Il ne prouve pas que la perte est faible ni que les paramètres sont optimaux.
- Pour vérifier une petite implémentation, une différence finie centrée permet de contrôler certaines dérivées. Il s'agit d'une vérification numérique, pas de l'algorithme d'entraînement. Comparez avec une tolérance : une perturbation trop petite peut entraîner une perte de précision par soustraction de nombres proches en virgule flottante.

Application récursive de la règle de la chaîne

$$\frac{\partial \ell}{\partial u} = \frac{\partial z}{\partial u} \cdot \frac{\partial \ell}{\partial z}$$



Pour un chemin en aval : $\frac{\partial \ell}{\partial u} = \frac{\partial z}{\partial u}\frac{\partial \ell}{\partial z}$.

La rétropropagation décompose le calcul en étapes élémentaires et applique la règle de la chaîne à chacune d'elles.

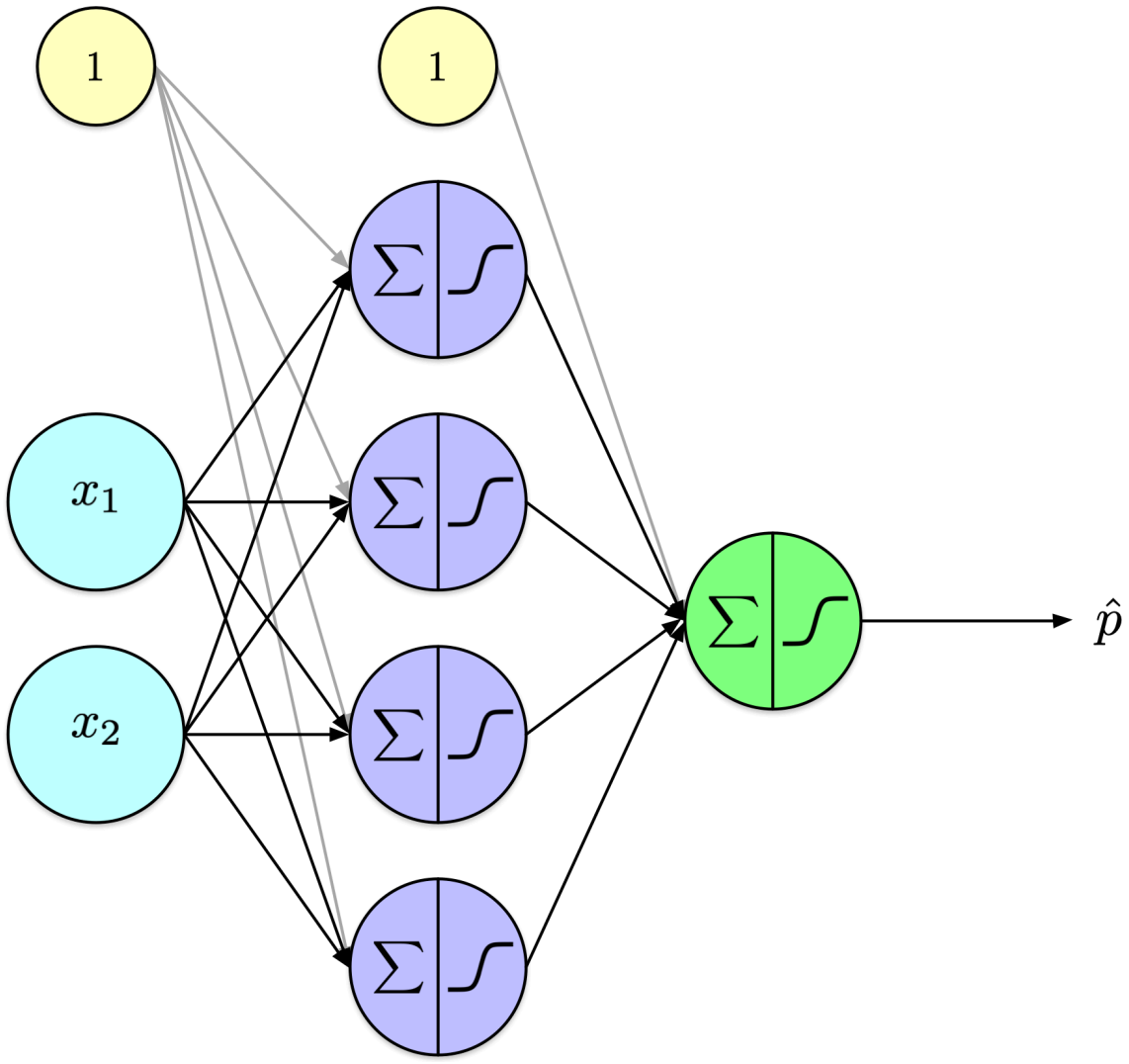
Le bloc rose calcule une sortie z à partir d'entrées comprenant u et v. L'entrée u peut être un poids, un biais ou une activation intermédiaire. La perte d'un exemple est ici

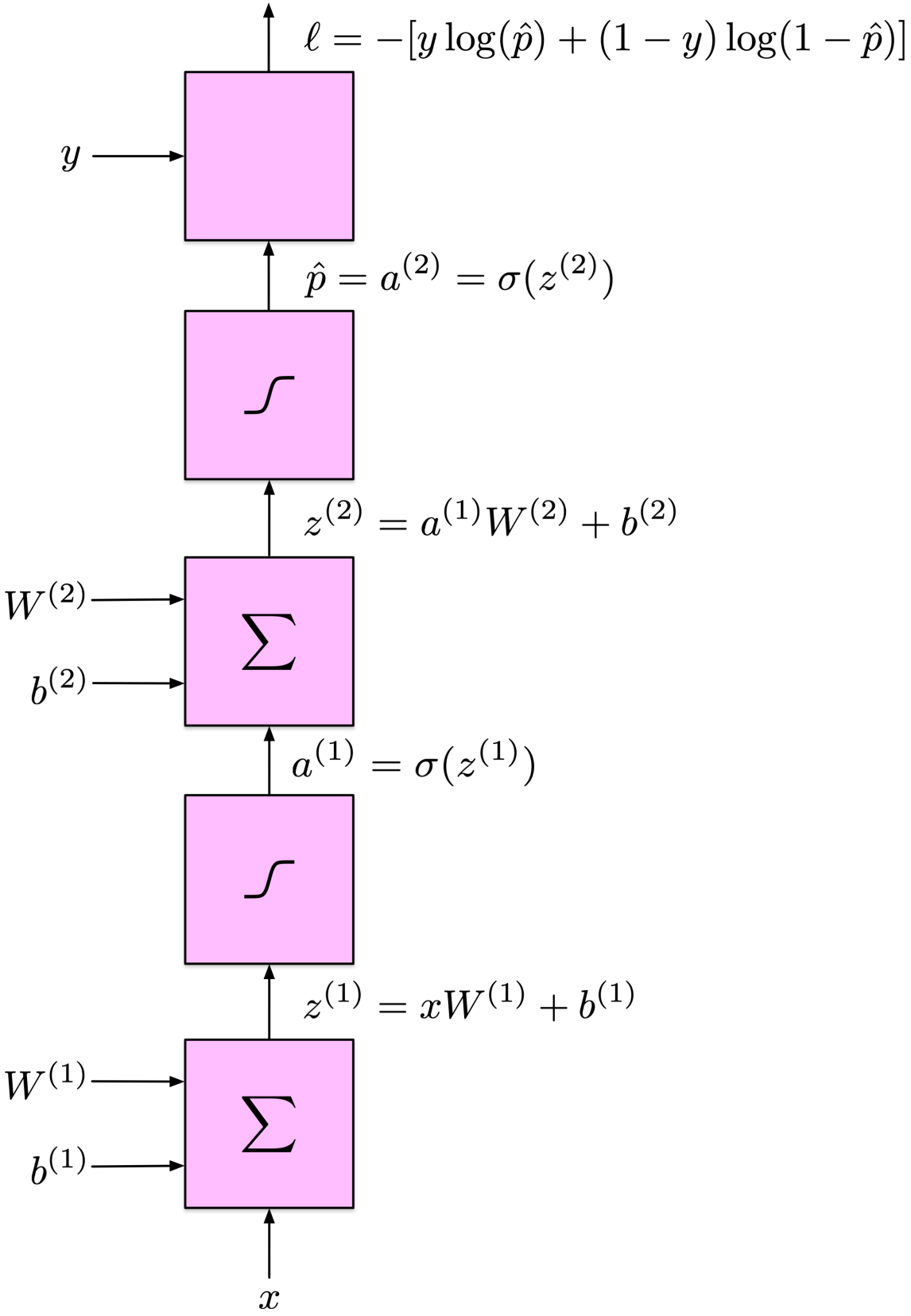
notée ℓ.

La dérivée $\partial \ell / \partial u$ mesure la sensibilité locale de la perte à u. Son signe indique l'effet d'une petite augmentation de u et sa valeur absolue mesure la sensibilité. Elle ne détermine pas directement une taille de mise à jour appropriée pour le paramètre.

Si $\partial \ell / \partial z$ est déjà connue, sa multiplication par la dérivée locale $\partial z / \partial u$ donne la contribution passant par z. Le produit affiché donne la dérivée totale par rapport à u uniquement si toute l'influence de u sur la perte passe par z. Si u alimente aussi d'autres opérations, leurs contributions doivent être ajoutées. Un exemple avec embranchement suivra l'exemple à un seul chemin.

Graphe de calcul





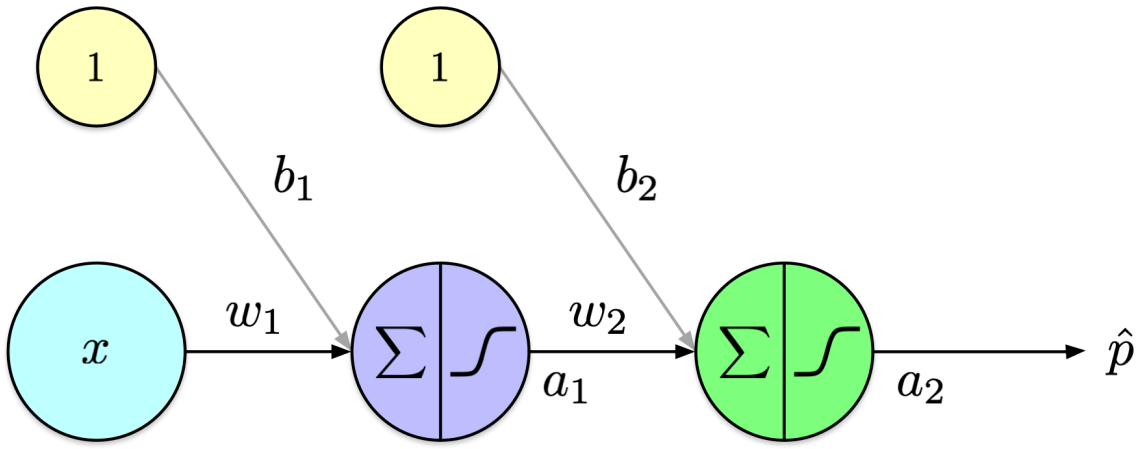
Un **perceptron à deux couches** et son **graphe de calcul** (*computational graph*).

Le schéma de gauche montre un réseau particulier, tandis que celui de droite regroupe les opérations par couche. À droite, x est un vecteur ligne : le même schéma de calcul convient donc à différents nombres d'unités dans chaque couche.

$W^{(l)}$ désigne les matrices de poids, $b^{(l)}$ les vecteurs de biais, $z^{(l)}$ les vecteurs de préactivations et $a^{(l)}$ les vecteurs d'activations associés à la couche l.

Le graphe représente la perte ℓ d'un exemple. Sa sortie sigmoïde est la probabilité prédite $\hat{p}$. La moyenne des pertes individuelles sur les exemples d'entraînement donne J. Les mêmes calculs s'appliquent à un lot, chaque exemple occupant une ligne.

Entrée scalaire et une unité cachée

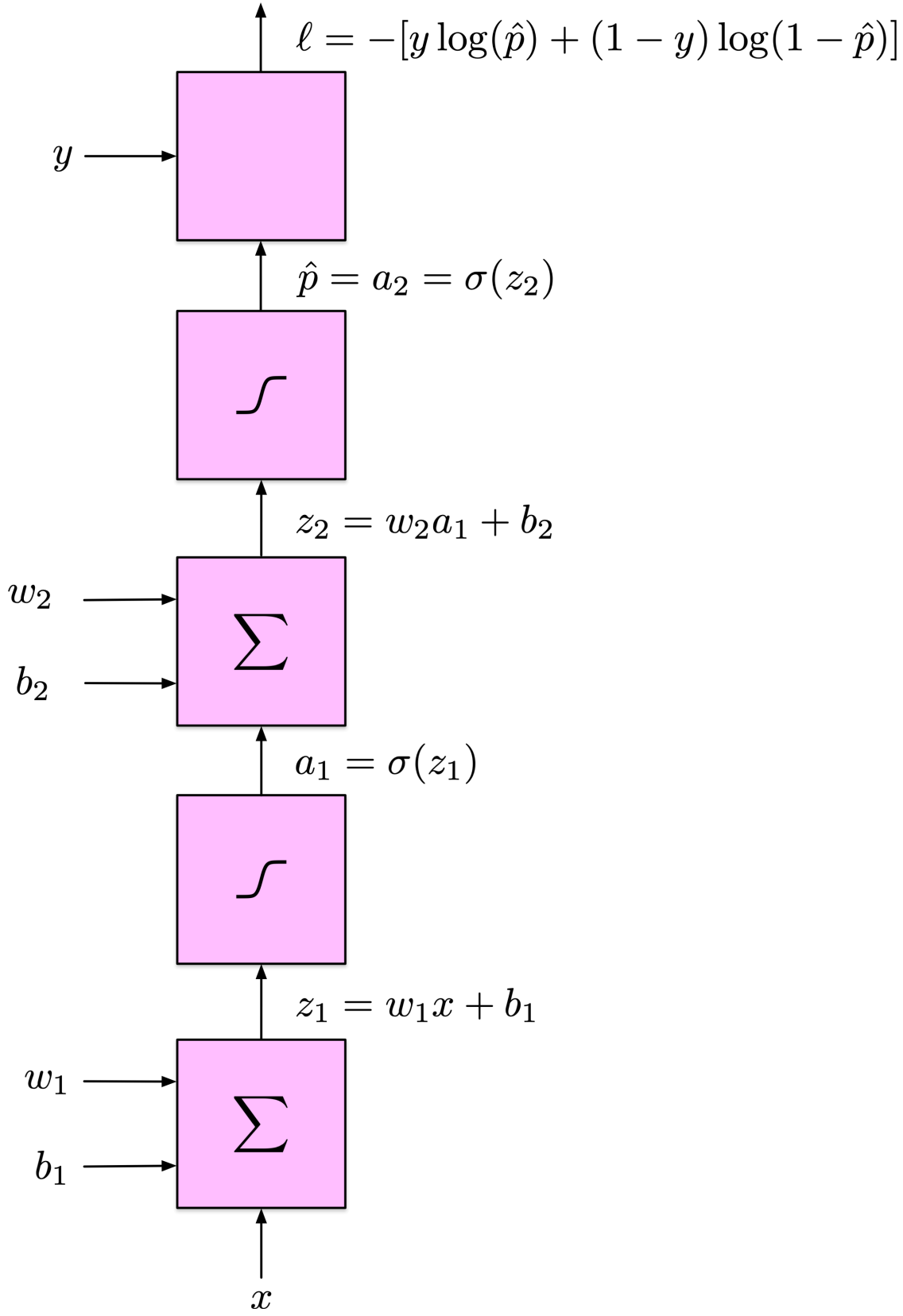


Soit

$$\ell = -\Big[y \log(\hat{p}) + (1-y) \log(1-\hat{p})\Big]$$

$$\hat{p} = a_2 = \sigma(z_2), \quad z_2 = w_2 \cdot a_1 + b_2$$

$$a_1 = \sigma(z_1), \quad z_1 = w_1 \cdot x + b_1$$



Nous commençons par une entrée scalaire et une unité par couche pour rendre chaque étape de la règle de la chaîne visible, sans introduire l'algèbre matricielle ni plusieurs indices d'unités. La couche cachée exige tout de même de propager la sensibilité de la perte à travers un calcul supplémentaire. Les réseaux plus grands suivent ces mêmes

principes et additionnent les contributions lorsqu'une activation alimente plusieurs unités.

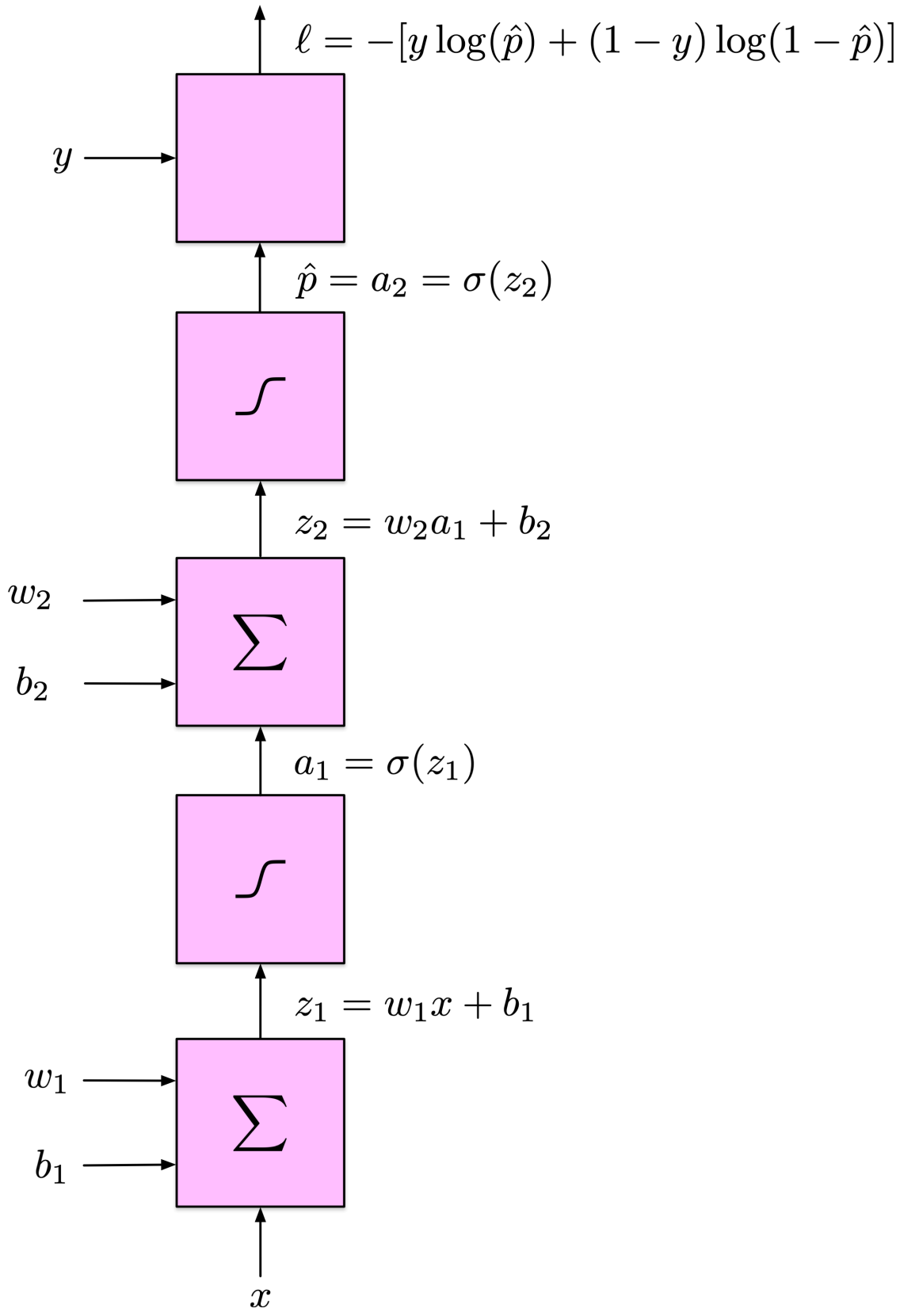
Le réseau comporte une entrée scalaire, une unité cachée et une unité de sortie. Chaque couche de traitement possède donc un poids, un biais, une préactivation et une activation. Nous notons ces valeurs uniques w_l, b_l, z_l, a_l. Ainsi, a_1 est l'unique composante de $a^{(1)}$ et z_1 celle de $z^{(1)}$. L'indice inférieur désigne ici la couche puisqu'il n'y a qu'une unité par couche.

Les deux schémas représentent le même réseau scalaire. Le graphe de calcul de droite sépare le calcul affine de chaque unité de son activation sigmoïde et inclut la perte. Dans le schéma du réseau, les entrées constantes de valeur 1 apportent b_1 et b_2 par leurs connexions respectives. L'activation cachée est a_1 et celle de sortie est $a_2 = \hat{p}$.

Le lien avec les transformations complètes des couches est $a_1 = f_1(x)$ et $\hat{p} = a_2 = f_2(a_1)$. Pour un exemple, $a^{(l)}$ et $z^{(l)}$ décrivent les quantités de la couche. Les matrices $A^{(l)}$ et $Z^{(l)}$ regroupent ces quantités pour plusieurs exemples stockés en lignes.

Dérivées

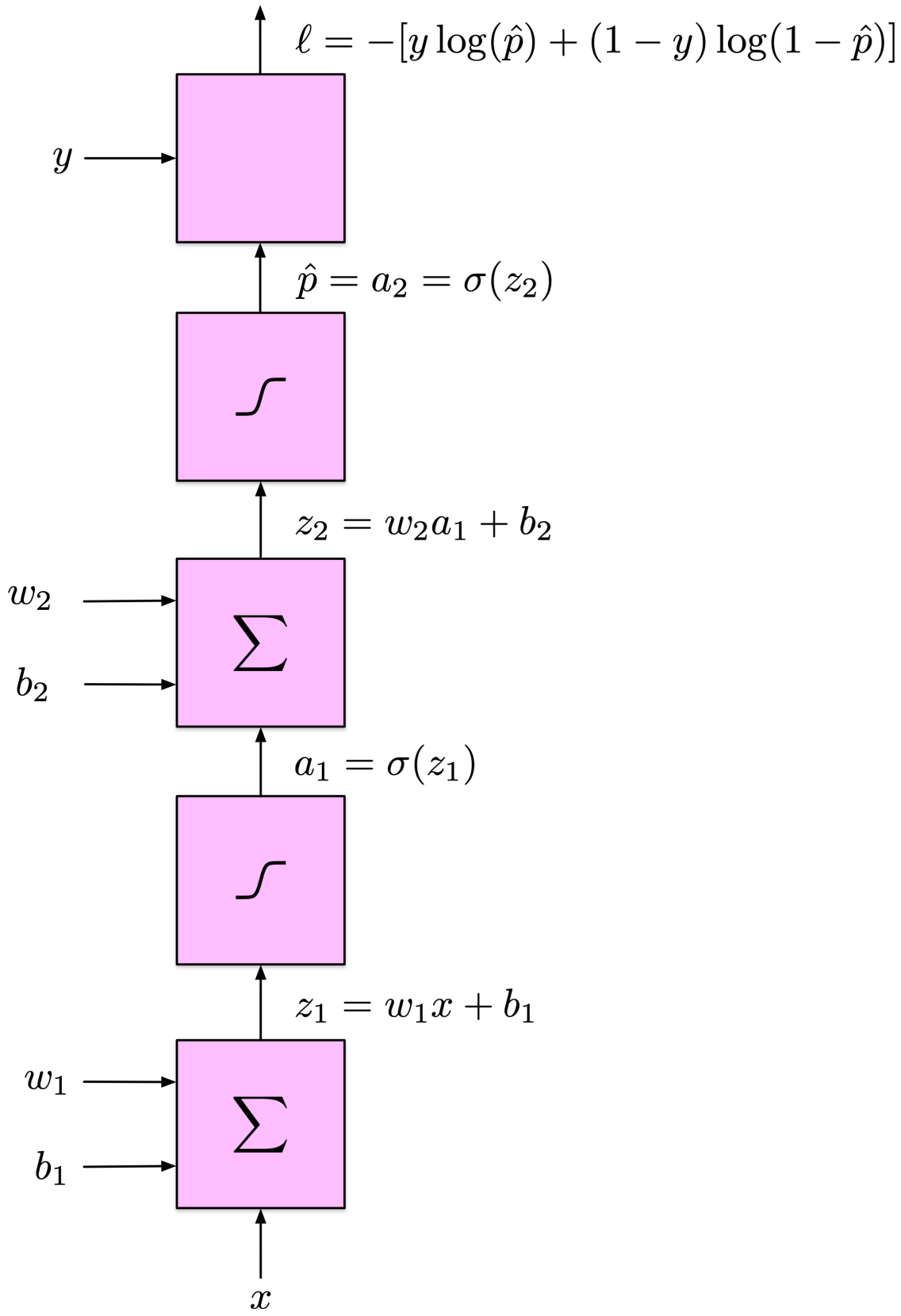
$$
\begin{array}{ccc}
 & \dfrac{\partial \ell}{\partial \hat{p}} & \\
 & \dfrac{\partial \hat{p}}{\partial z_2} & \\
\dfrac{\partial z_2}{\partial w_2}, & \dfrac{\partial z_2}{\partial b_2}, & \dfrac{\partial z_2}{\partial a_1}, \\
 & \dfrac{\partial a_1}{\partial z_1}, & \\
\dfrac{\partial z_1}{\partial w_1}, & \dfrac{\partial z_1}{\partial b_1}, & \dfrac{\partial z_1}{\partial x},
\end{array}
$$



Dérivées

Dérivée de la perte par rapport à $\hat{p}$:

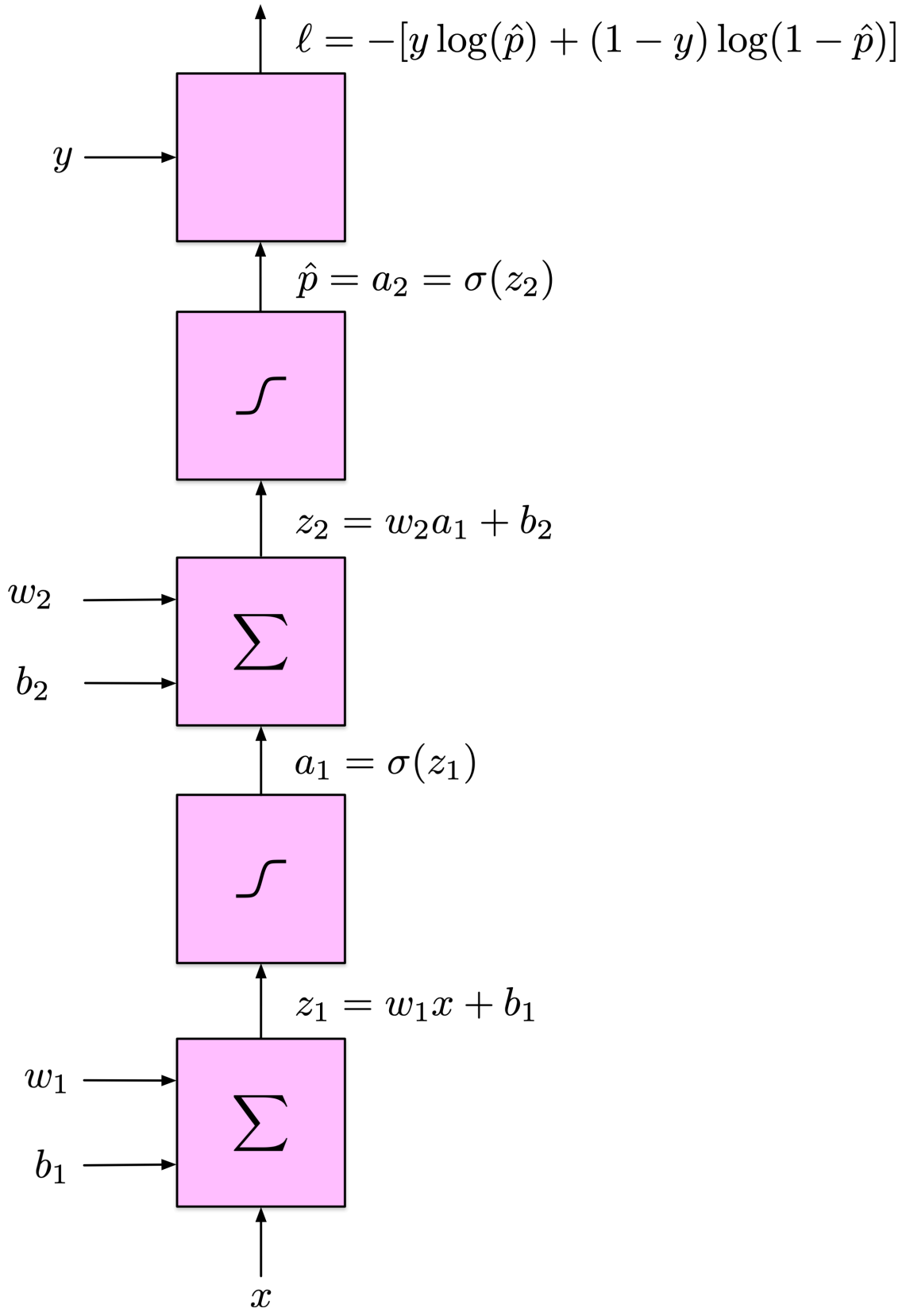
$$\frac{\partial \ell}{\partial \hat{p}} = -\left(\frac{y}{\hat{p}} - \frac{1-y}{1-\hat{p}}\right)$$



Dérivées

Dérivée de $\hat{p}$ par rapport à z_2 :

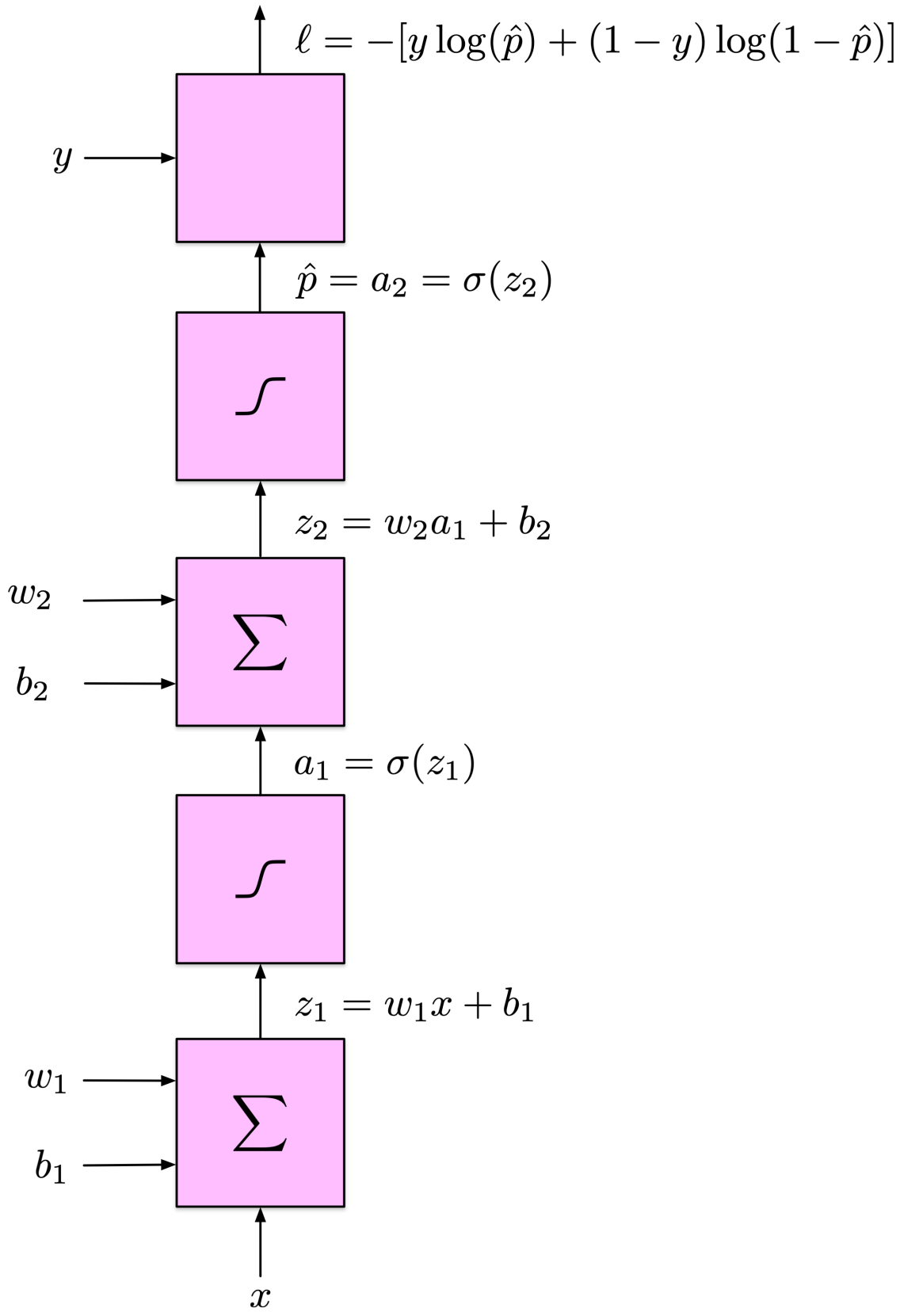
$$\frac{\partial \hat{p}}{\partial z_2} = \sigma'(z_2) = \hat{p}(1-\hat{p})$$



Dérivées

Dérivées de $z_2 = w_2 a_1 + b_2$:

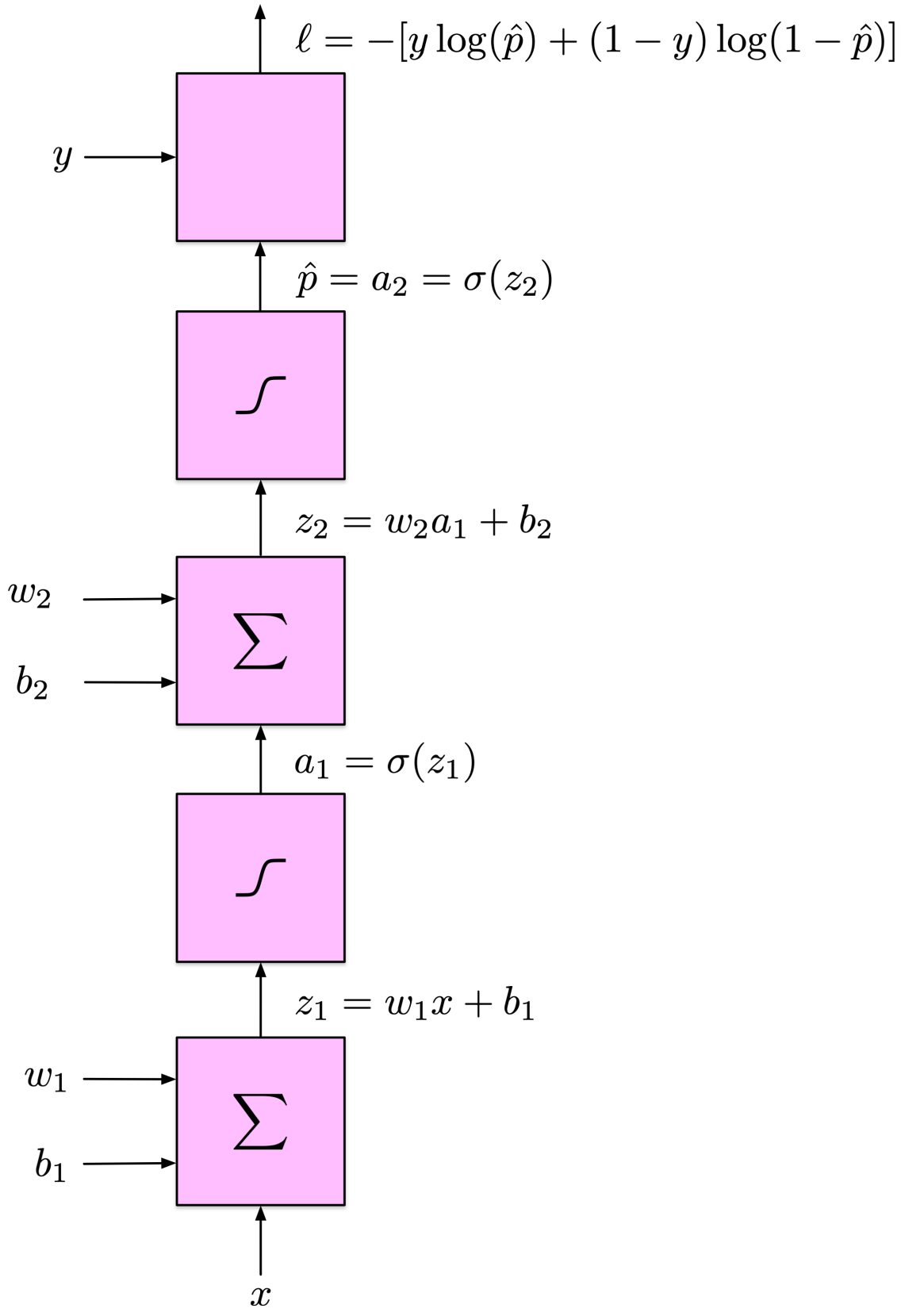
$$\frac{\partial z_2}{\partial w_2} = a_1, \quad \frac{\partial z_2}{\partial b_2} = 1, \quad \frac{\partial z_2}{\partial a_1} = w_2$$



Dérivées

Dérivée de $a_1 = \sigma(z_1)$:

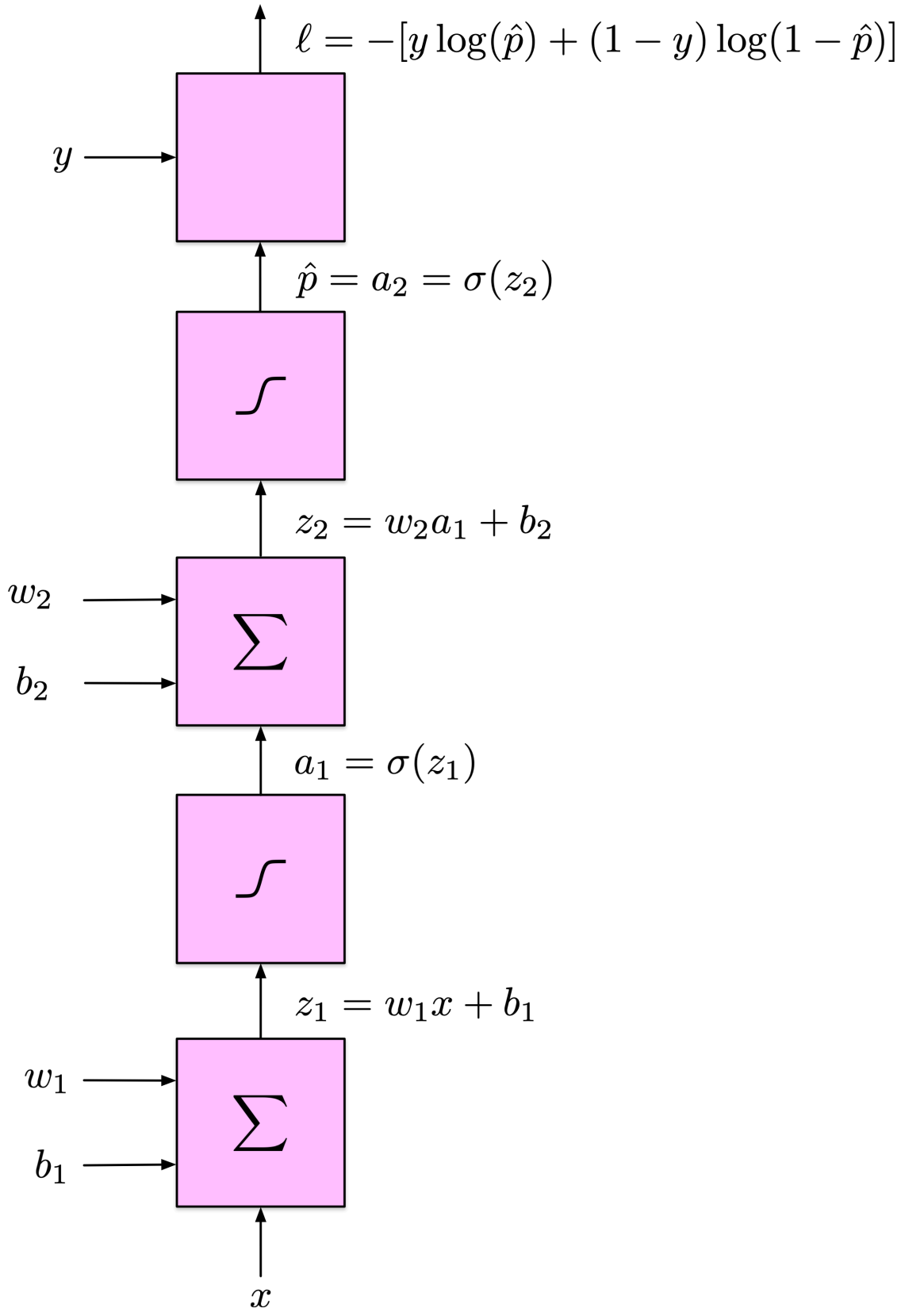
$$\frac{\partial a_1}{\partial z_1} = \sigma'(z_1) = a_1(1 - a_1)$$



Dérivées

Dérivées de $z_1 = w_1 x + b_1$:

$$\frac{\partial z_1}{\partial w_1} = x, \quad \frac{\partial z_1}{\partial b_1} = 1, \quad \frac{\partial z_1}{\partial x} = w_1$$



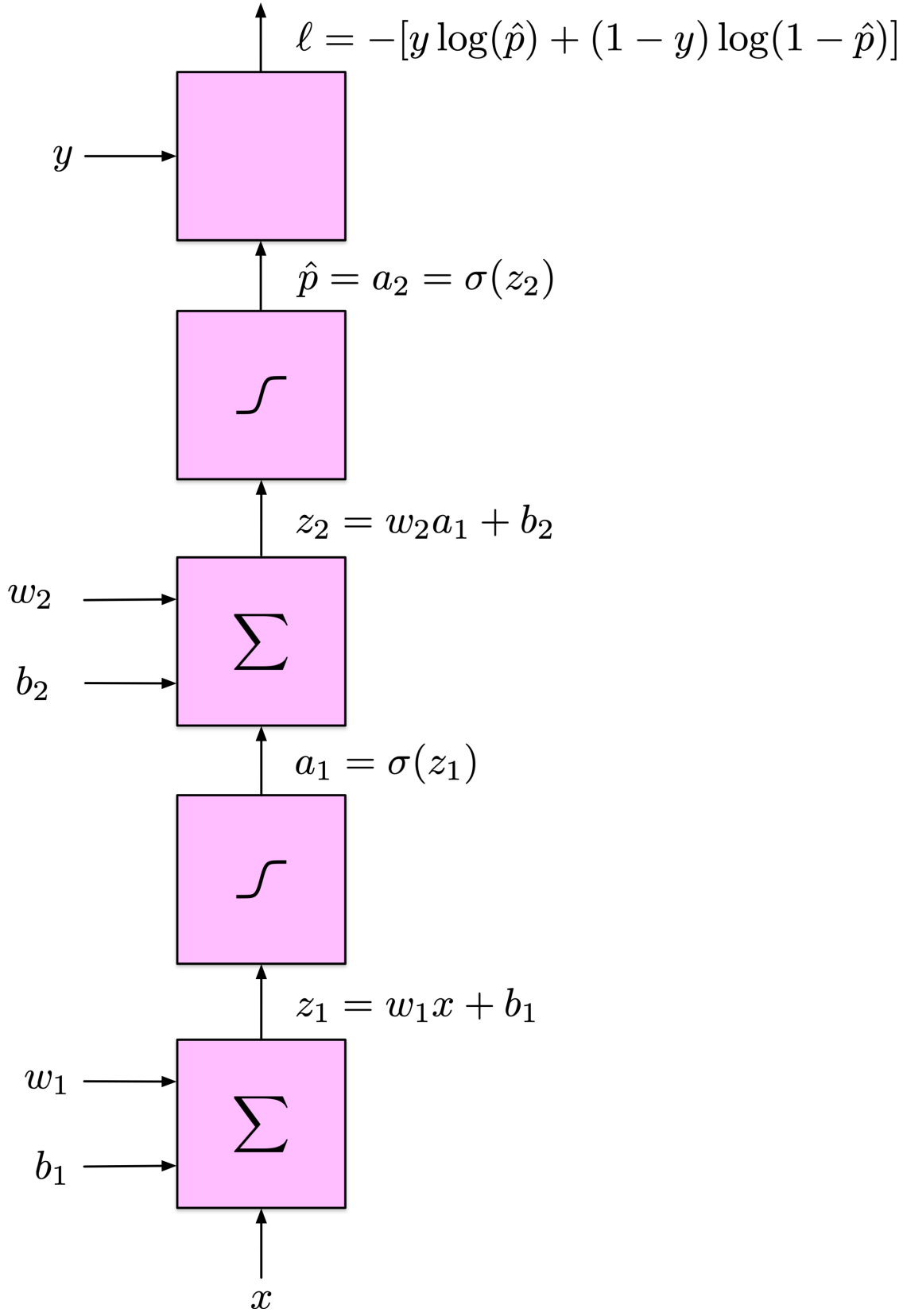
Dérivées combinées

Pour w_2 :

$$\frac{\partial \ell}{\partial w_2} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2} = \left[- \left(\frac{y}{\hat{p}} - \frac{1-y}{1-\hat{p}} \right) \right] \cdot (\hat{p}(1-\hat{p})) \cdot a_1$$

Après simplification :

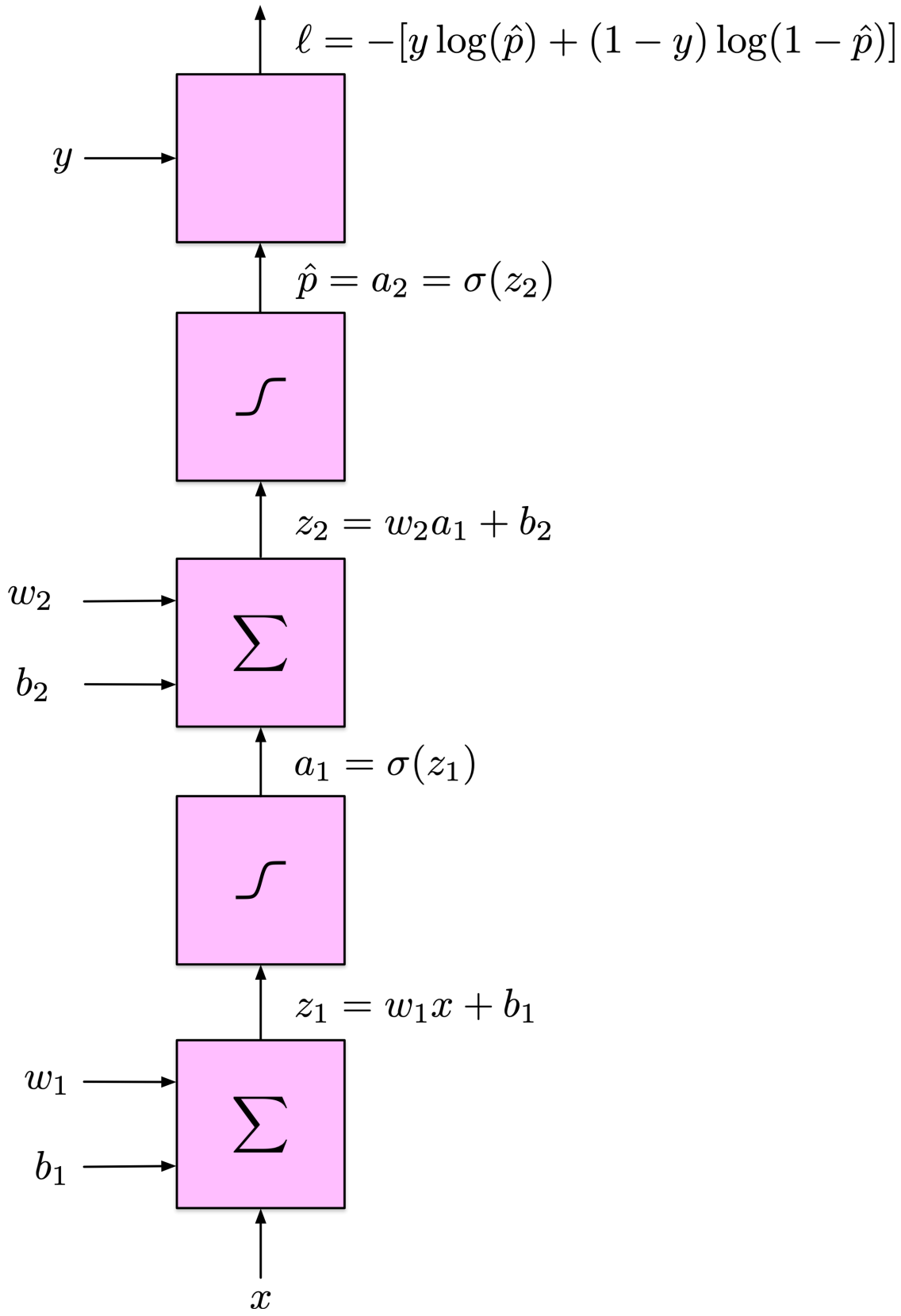
$$\frac{\partial \ell}{\partial w_2} = (\hat{p} - y) a_1$$



Dérivées combinées

Pour b_2 :

$$\frac{\partial \ell}{\partial b_2} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} = (\hat{p} - y) \cdot 1 = \hat{p} - y$$



Dérivées combinées

Pour w_1 :

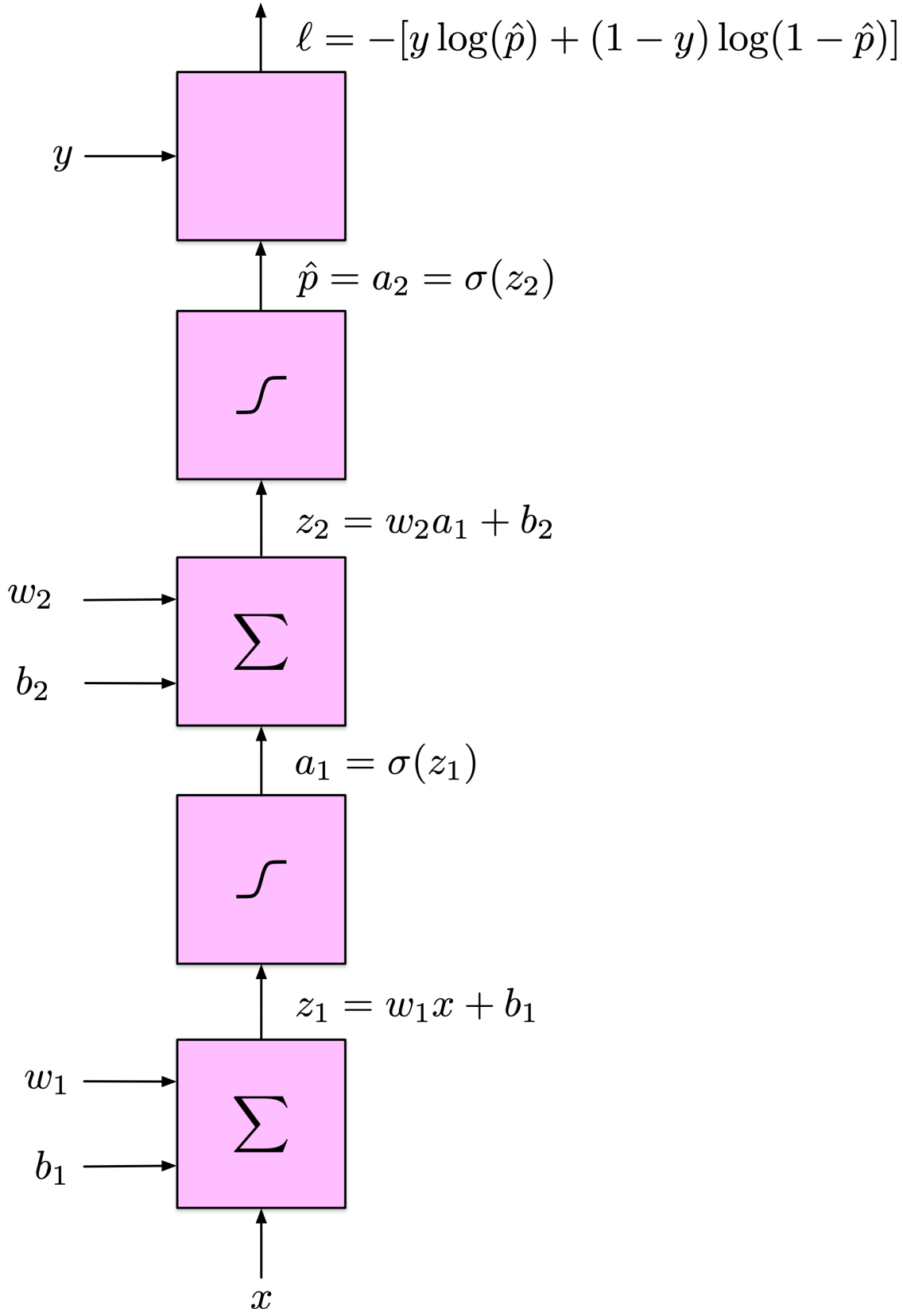
$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

En remplaçant les dérivées locales :

$$= \left[- \left(\frac{y}{\hat{p}} - \frac{1-y}{1-\hat{p}} \right) \right] \cdot (\hat{p}(1-\hat{p})) \cdot w_2 \cdot (a_1 \left(1 - a_1\right)) \cdot x$$

Après simplification :

$$\frac{\partial \ell}{\partial w_1} = (\hat{p} - y) w_2 \left(a_1 \left(1 - a_1\right)\right) x$$



Dérivées combinées

Pour b_1 :

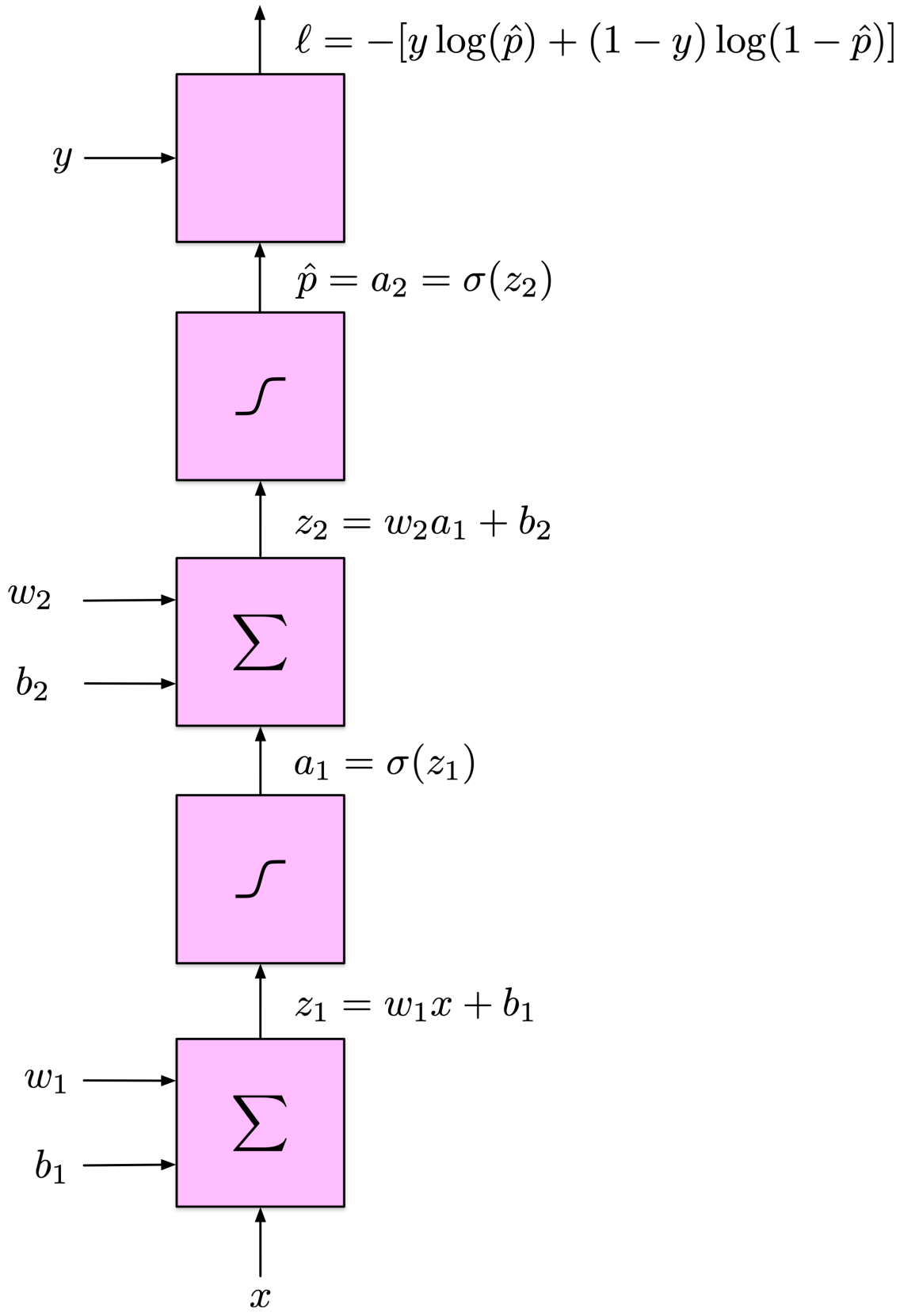
$$\frac{\partial \ell}{\partial b_1} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1}$$

En remplaçant les dérivées locales :

$$= (\hat{p} - y) w_2 \left(a_1 \left(1 - a_1\right)\right) \cdot 1$$

Après simplification :

$$\frac{\partial \ell}{\partial b_1} = (\hat{p} - y) w_2 \left(a_1 \left(1 - a_1\right)\right)$$



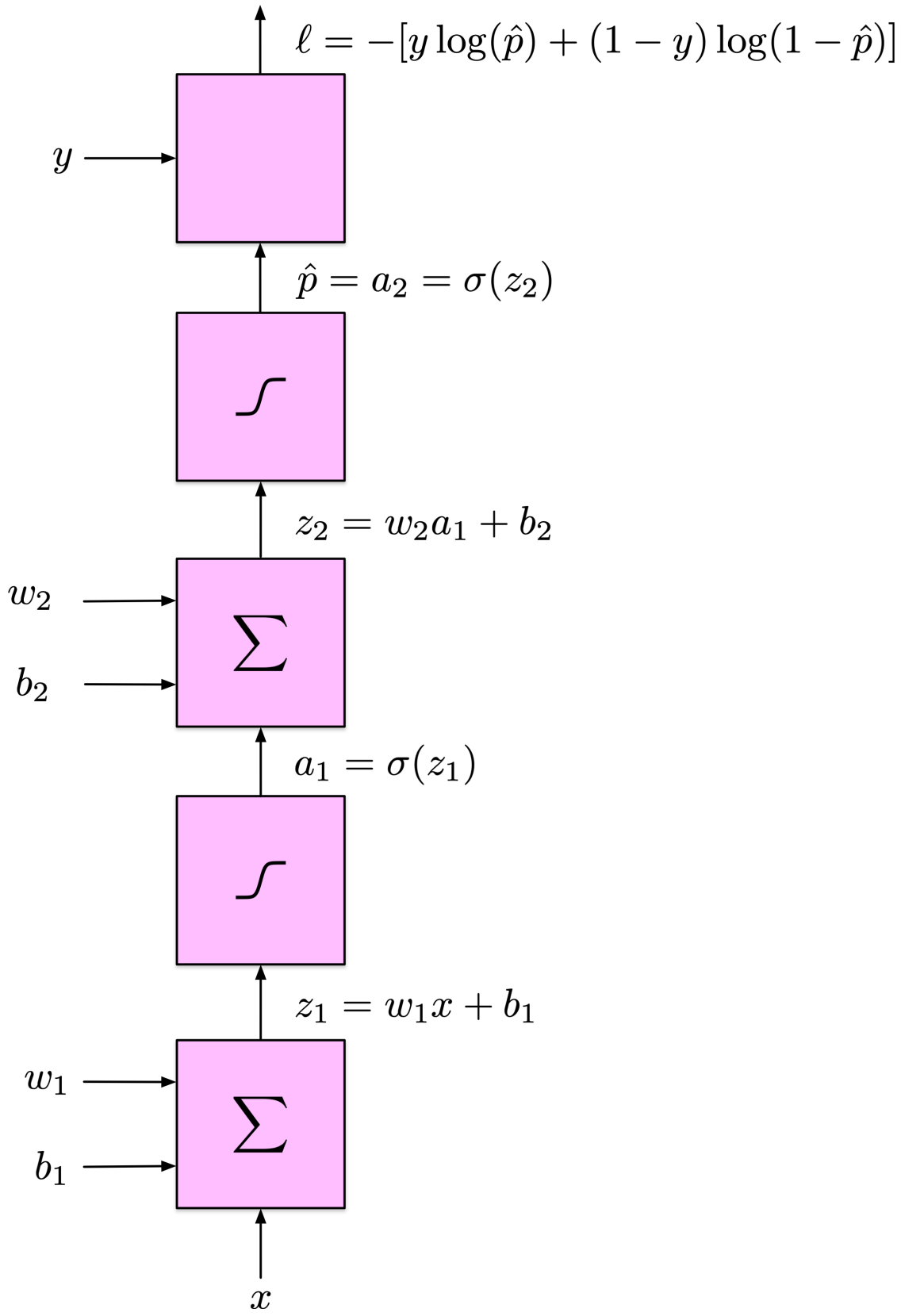
Dérivées principales

$$\frac{\partial \ell}{\partial w_2} = (\hat{p} - y) a_1 \tag{1}$$

$$\frac{\partial \ell}{\partial b_2} = \hat{p} - y \tag{2}$$

$$\frac{\partial \ell}{\partial w_1} = (\hat{p} - y) w_2 \left(a_1 \left(1 - a_1\right)\right) x \tag{3}$$

$$\frac{\partial \ell}{\partial b_1} = (\hat{p} - y) w_2 \left(a_1 \left(1 - a_1\right)\right) \tag{4}$$



Observation 1 : Le calcul des gradients utilise les activations des couches, en particulier a_1 et $\hat{p} = a_2$.

Exploration

In [3]:
```python
import math
import random

random.seed(42)

def sigma(x):
    return 1 / (1 + math.exp(-x))

alpha = 0.1

def init():

    global w1, w2, b1, b2

    w1 = random.random()
    w2 = random.random()
    b1 = 0
    b2 = 0
```

Ce programme implémente directement les équations précédentes. Son architecture fixe et ses variables scalaires nommées rendent chaque valeur intermédiaire, dérivée et mise à jour de paramètre facile à examiner. L'implémentation NumPy présentée ensuite organise ces mêmes opérations en boucles et en calculs matriciels pour traiter plusieurs unités et des lots d'exemples.

Les quatre paramètres appris sont w_1, b_1, w_2, b_2, qui correspondent à `w1`, `b1`, `w2` et `b2` dans le code.

Propagation avant

In [4]:
```python
def forward():

    global z1, a1, z2, a2, loss, p_hat

    z1 = w1 * x + b1
    a1 = sigma(z1)

    z2 = w2 * a1 + b2
    a2 = sigma(z2)

    p_hat = a2

    loss = -(y * math.log(p_hat) + (1-y) * math.log(1 - p_hat))
```

Le passage avant calcule et conserve `z1`, `a1`, `z2` et `a2`. La variable `p_hat = a2` est la probabilité de sortie, et `loss` est la perte ℓ d'un exemple. La déclaration `global` permet à la fonction de conserver ces valeurs pour le calcul arrière. Ce programme volontairement petit utilise des variables partagées pour que chaque ligne puisse être comparée directement aux équations scalaires.

Propagation arrière

In [5]:
```
def backward():

    grad_loss_w2 = (p_hat - y) * a1
    grad_loss_b2 = p_hat - y

    grad_loss_w1 = (p_hat - y) * w2 * (a1 * (1-a1)) * x
    grad_loss_b1 = (p_hat - y) * w2 * (a1 * (1-a1))

    return grad_loss_w1, grad_loss_b1, grad_loss_w2, grad_loss_b2
```

Le passage arrière calcule les quatre dérivées des paramètres à partir des valeurs du passage avant. Il ne modifie aucun paramètre. Toutes les dérivées doivent être évaluées aux mêmes valeurs des paramètres. En particulier, les dérivées de la couche cachée utilisent le poids de sortie `w2` qui a produit la prédiction courante.

Mise à jour par gradient

In [6]:
```
def step(gradients):

    global w1, b1, w2, b2

    grad_loss_w1, grad_loss_b1, grad_loss_w2, grad_loss_b2 = gradients

    w1 = w1 - alpha * grad_loss_w1
    b1 = b1 - alpha * grad_loss_b1
    w2 = w2 - alpha * grad_loss_w2
    b2 = b2 - alpha * grad_loss_b2
```

La rétropropagation calcule les dérivées et la descente de gradient les utilise pour mettre les paramètres à jour. La fonction `step` implémente la mise à jour de la troisième diapositive « Idée conceptuelle ». Bien que les affectations occupent des lignes successives, chaque mise à jour utilise un gradient calculé avant toute modification des paramètres.

Entraînement

In [7]:
```
init()

x = 3.14
y = 1

forward()
print(f"Avant : p_hat = {p_hat:.2}, perte = {loss:.2}")

for i in range(500):
    forward()
    gradients = backward()
```

```
    step(gradients)

forward()
print(f"Après : p_hat = {p_hat:.2}, perte = {loss:.2}")
```

```
Avant : p_hat = 0.51, perte = 0.68
Après : p_hat = 0.99, perte = 0.01
```

Cette démonstration entraîne le réseau plusieurs fois sur un seul exemple pour rendre explicites le calcul de chaque dérivée et la mise à jour correspondante des paramètres. Elle ne mesure pas la généralisation. Ni l'architecture ni l'entropie croisée binaire ne limitent l'entraînement à un seul exemple. Une boucle peut traiter plusieurs exemples, et l'implémentation NumPy présentée ensuite les traite par lots.

Les variables `p_hat` et `loss` correspondent à $\hat{p}$ et ℓ. Fixer `y=0`, changer `x` ou ajuster `alpha` permet de mener de petites expériences avec ce même modèle à quatre paramètres.

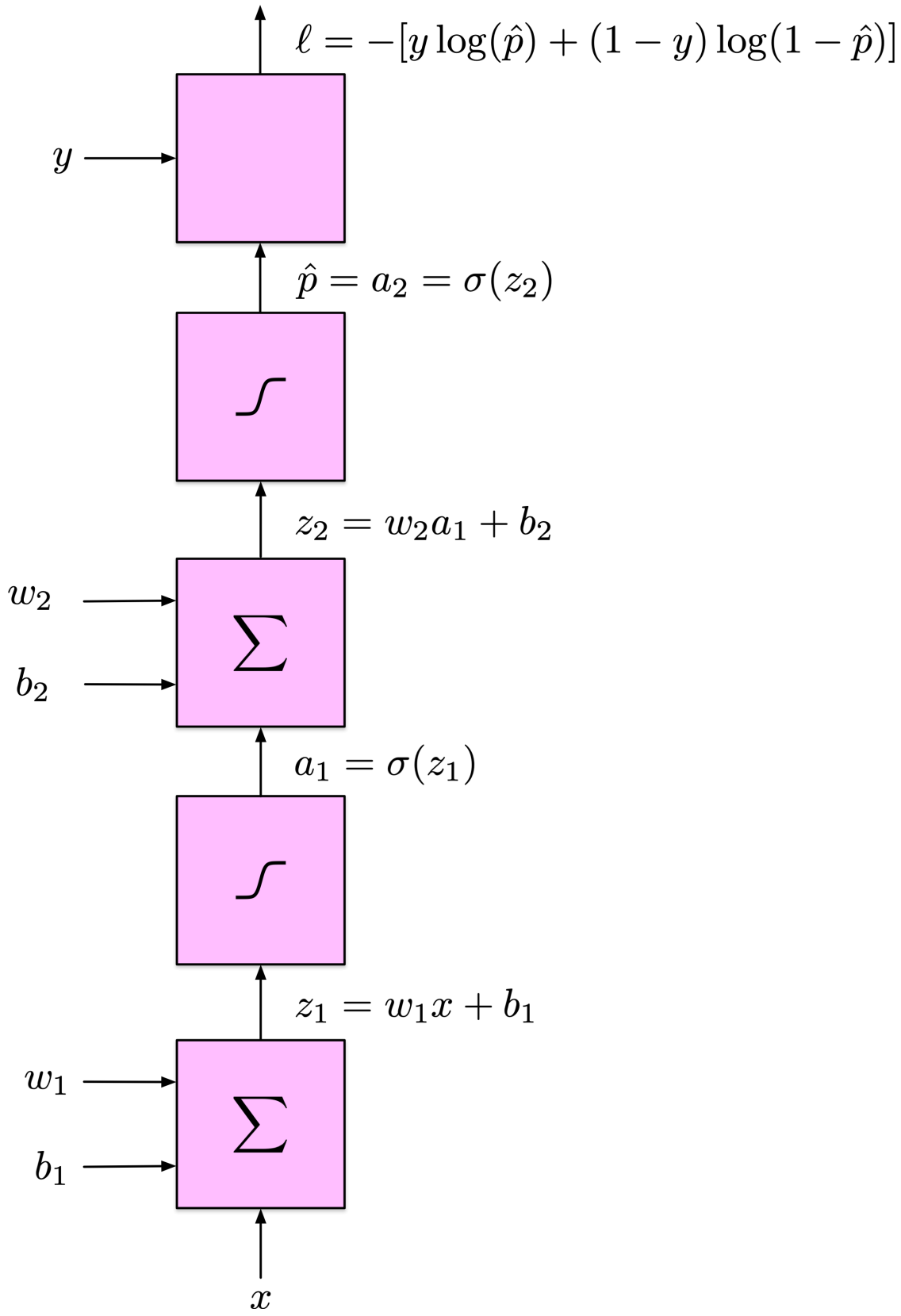
Dérivées principales

$$\frac{\partial \ell}{\partial w_2} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2} \tag{5}$$

$$\frac{\partial \ell}{\partial b_2} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} \tag{6}$$

$$\frac{\partial \ell}{\partial w_1} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} \tag{7}$$

$$\frac{\partial \ell}{\partial b_1} = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} \tag{8}$$



Observation 2 : Certaines dérivées, comme $\frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2}$, sont calculées plusieurs fois.

Ces répétitions s'expliquent par le fait que les chaînes de dérivées suivent le même chemin dans le graphe de calcul.

Signaux d’erreur

Soit

$$\delta_2 = \frac{\partial \ell}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z_2} \tag{9}$$

$$\delta_1 = \delta_2 \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \tag{10}$$

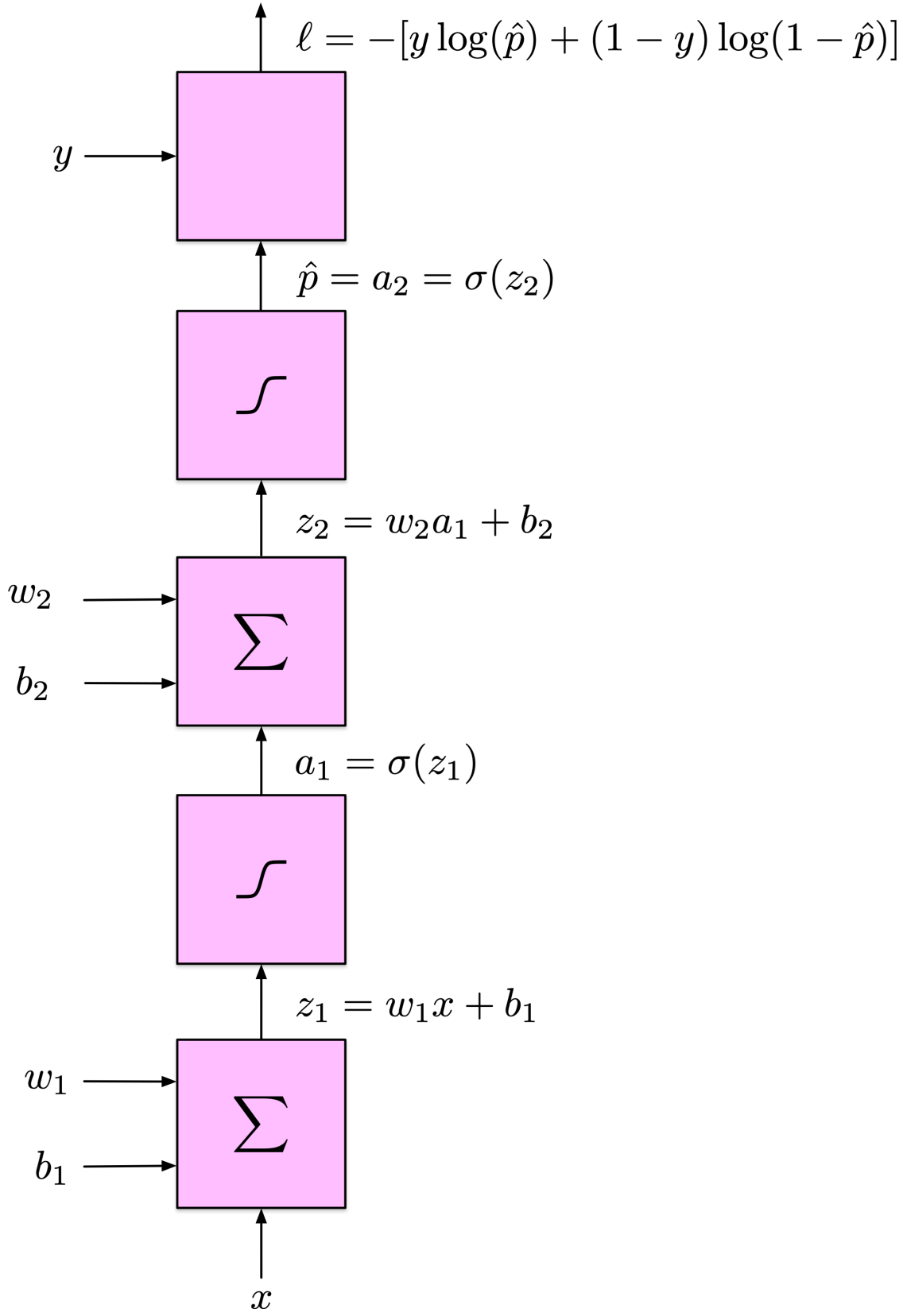
Réécrivons

$$\frac{\partial \ell}{\partial w_2} = \delta_2 \cdot \frac{\partial z_2}{\partial w_2} \tag{11}$$

$$\frac{\partial \ell}{\partial b_2} = \delta_2 \cdot \frac{\partial z_2}{\partial b_2} \tag{12}$$

$$\frac{\partial \ell}{\partial w_1} = \delta_1 \cdot \frac{\partial z_1}{\partial w_1} \tag{13}$$

$$\frac{\partial \ell}{\partial b_1} = \delta_1 \cdot \frac{\partial z_1}{\partial b_1} \tag{14}$$



Le **signal d'erreur** (*error signal*) $\delta_l = \partial \ell / \partial z_l$ mesure la sensibilité de la perte à la **préactivation** d'une unité. Son indice désigne la couche.

Les produits répétés ont une interprétation utile : δ_2 mesure la sensibilité de la perte à z_2 et δ_1 celle à z_1. Ce sont les signaux d'erreur. Une unité cachée n'a pas besoin de sa

propre valeur cible : son signal d'erreur décrit comment une petite variation de sa préactivation modifierait la perte finale.

Pour une sortie sigmoïde et l'entropie croisée binaire, $\delta_2 = \hat{p} - y$. Cette différence simple résulte de la simplification des dérivées déjà calculées. Elle est propre à cette combinaison activation–perte : un signal d'erreur n'est pas toujours la différence entre une prédiction et une cible. Le signal d'erreur caché est $\delta_1 = \delta_2 w_2 a_1 (1 - a_1)$.

Les dérivées des paramètres découlent des mêmes signaux : $\partial \ell / \partial w_l = \delta_l a_{l-1}$ et $\partial \ell / \partial b_l = \delta_l$, avec $a_0 = x$. Réutiliser ces quantités évite de recalculer les mêmes produits de dérivées pour différents paramètres.

Propagation arrière avec signaux d'erreur

In [8]:
```
def backward_with_signals():

    delta2 = p_hat - y
    delta1 = delta2 * w2 * a1 * (1 - a1)

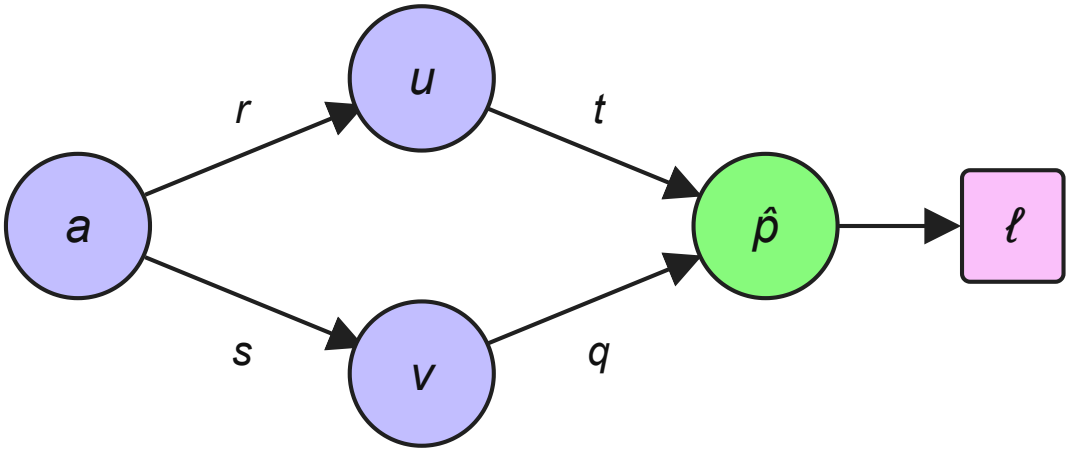
    grad_loss_w2 = delta2 * a1
    grad_loss_b2 = delta2
    grad_loss_w1 = delta1 * x
    grad_loss_b1 = delta1

    return grad_loss_w1, grad_loss_b1, grad_loss_w2, grad_loss_b2
```

Cette fonction renvoie les quatre mêmes dérivées que `backward()`. Le calcul est organisé autrement : on calcule le signal d'erreur de sortie, on le propage vers l'unité cachée, puis on réutilise chaque signal pour le poids et le biais de son unité. La même fonction `step` peut utiliser l'un ou l'autre ensemble de dérivées.

Le passage avant conserve les activations et les préactivations. Le passage arrière les réutilise et calcule les signaux d'erreur de la sortie vers l'entrée. L'ajout de couches répète ce schéma de calcul plutôt que d'exiger une nouvelle dérivation de chaque chemin complet.

Une activation alimente deux unités



$$z_u = ra + b_u, \qquad u = \sigma(z_u)$$

$$z_v = sa + b_v, \qquad v = \sigma(z_v)$$

$$\hat{p} = \sigma(tu + qv + b_{\text{out}})$$

Avec $\delta_u = \partial \ell / \partial z_u$ et $\delta_v = \partial \ell / \partial z_v$:

$$\frac{\partial \ell}{\partial a} = \underbrace{r\delta_u}_{\text{par } u} + \underbrace{s\delta_v}_{\text{par } v}.$$

Multiplier le long d'un chemin et additionner les contributions des différents chemins en aval.

L'activation cachée partagée a influence maintenant la perte par deux unités suivantes, u et v, qui influencent toutes deux la même probabilité de sortie. Modifier a change à la fois z_u et z_v : la règle de la chaîne inclut donc les deux effets. Les signaux en aval δ_u et δ_v ont déjà été calculés à partir de la perte de sortie. Leur multiplication par r et s donne les contributions respectives à $\partial \ell / \partial a$.

Si $a = \sigma(z_a)$, son propre signal d'erreur est

$$\delta_a = \frac{\partial \ell}{\partial z_a} = (r\delta_u + s\delta_v)\, a(1 - a).$$

Rassemblez toutes les contributions avant de poursuivre vers l'amont. Elles peuvent se renforcer ou s'opposer. On les additionne sans calculer de moyenne sur les unités en aval. La moyenne sur les exemples d'entraînement est une opération distincte. Cette accumulation est une propriété générale de la différentiation en mode inverse (Baydin et al. 2017).

Embranchement : Python

In [9]:
```
a = sigma(0.0)
r, s = 2.0, -1.0
t, q = 1.0, 1.0
u = sigma(r * a - 1.0)
v = sigma(s * a + 0.5)
p_hat = sigma(t * u + q * v - 1.0)
y = 1

delta_out = p_hat - y
delta_u = delta_out * t * u * (1 - u)
delta_v = delta_out * q * v * (1 - v)

through_u = r * delta_u
through_v = s * delta_v
delta_a = (through_u + through_v) * a * (1 - a)

print(f"Contributions : {through_u:.3f} + {through_v:.3f}")
print(f"d_loss/da: {through_u + through_v:.3f}")
print(f"delta_a: {delta_a:.5f}")
```

```
Contributions : -0.250 + 0.125
d_loss/da: -0.125
delta_a: -0.03125
```

Les biais sont $b_u = -1$, $b_v = 0.5$ et $b_{\text{out}} = -1$. Ces choix donnent $a = u = v = \hat{p} = 0.5$, ce qui simplifie l'arithmétique. Comme $y = 1$, le signal de sortie est $\delta_{\text{out}} = -0.5$ et les deux signaux cachés en aval valent -0.125.

La contribution passant par u vaut $2(-0.125) = -0.25$. Celle passant par v vaut $(-1)(-0.125) = +0.125$. Leur somme vaut -0.125 : les deux effets s'annulent partiellement. La multiplication par $a(1-a) = 0.25$ donne $\delta_a = -0.03125$.

Cet exemple étend le calcul arrière d'un chemin unique au cas d'une activation partagée, sans introduire une nouvelle méthode d'optimisation. Dans le réseau général, `delta @ W_next.T` calcule ces sommes pondérées pour toutes les unités à la fois. La multiplication par la dérivée de l'activation produit ensuite les signaux d'erreur de la couche précédente.

Entraînement avec rétropropagation

1. **Initialiser** les poids et les biais.
2. **Propagation avant :** calculer les activations et conserver les valeurs intermédiaires.
3. **Calculer la perte** du lot courant.
4. **Propagation arrière :** calculer les gradients de tous les paramètres.
5. **Mise à jour par gradient :** modifier les paramètres à l'aide de ces gradients.
6. **Répéter** pour les lots et les époques restants.

La rétropropagation correspond à l'étape 4, le calcul efficace des gradients. L'optimiseur effectue l'étape 5. Notre optimiseur utilise la descente de gradient, tandis que d'autres

méthodes, comme Adam, exploitent les gradients autrement. La boucle d'entraînement s'arrête après un nombre choisi d'époques ou selon un critère d'arrêt, comme l'absence d'amélioration de la perte de validation.

Le programme scalaire traite un exemple à la fois. L'implémentation suivante traite un mini-lot à la fois et calcule la moyenne des gradients des exemples avant de mettre les paramètres à jour. Un lot contenant tout l'ensemble d'entraînement donne la descente de gradient par lot. Un lot d'un seul exemple donne la descente de gradient stochastique.

Rétropropagation sur un graphe de calcul

- Commencer à la perte : sa dérivée par rapport à elle-même est $\frac{\partial \ell}{\partial \ell} = 1$.
- Visiter les opérations dans **l'ordre inverse du calcul**.
- Pour chaque entrée u, **additionner les contributions de toutes ses utilisations en aval**:

$$\frac{\partial \ell}{\partial u} = \sum_{z \text{ utilise directement } u} \frac{\partial z}{\partial u} \frac{\partial \ell}{\partial z}.$$

Rassembler chaque contribution avant de propager le résultat plus loin vers l'amont.

Le passage arrière propage les sensibilités de la perte scalaire finale. Au nœud de perte lui-même, la sensibilité vaut un : une petite variation de la perte modifie cette même quantité d'exactement la même valeur. Il s'agit de la dérivée de la fonction identité. Elle initialise la règle de la chaîne, car multiplier la première dérivée locale par un la laisse inchangée. Cela ne signifie pas que la valeur de la perte est un.

Cette sensibilité initiale diffère du signal d'erreur de l'unité de sortie, $\partial \ell / \partial z_{\text{out}}$. Pour une sortie sigmoïde et l'entropie croisée binaire, ce signal est $\hat{p} - y$, comme établi précédemment. Notre implémentation manuelle le calcule directement. Un moteur général de graphe commence avec une sensibilité de un à la perte, puis remonte les opérations de perte et d'activation de sortie pour l'obtenir.

Pour un graphe d'opérations scalaires, la somme porte sur les sorties des opérations qui utilisent directement u. Chaque terme est une dérivée locale multipliée par une sensibilité en aval déjà calculée. Un seul chemin donne le produit de l'exemple scalaire. Plusieurs chemins donnent la somme de l'exemple avec embranchement.

Une implémentation peut initialiser les accumulateurs de dérivées à zéro, fixer celui de la perte à un et ajouter chaque contribution au lieu d'écraser une contribution précédente. L'ordre inverse du calcul garantit que les contributions en aval sont disponibles avant de poursuivre vers l'amont. Les valeurs qui n'influencent pas la perte ont une dérivée nulle.

Le symbole δ est réservé ici à la sensibilité de la perte à la préactivation d'une unité, $\partial \ell / \partial z$. Pour une entrée quelconque u du graphe, la dérivée s'écrit explicitement $\partial \ell / \partial u$.

Les bibliothèques modernes enregistrent les opérations pour automatiser ce processus. Notre implémentation NumPy utilise plutôt une séquence d'opérations de couches spécifiée manuellement et une boucle arrière, sans construire un moteur général de graphe (Baydin et al. 2017).

Initialisation

- Initialiser les **poids aléatoirement** pour briser la symétrie entre les unités.
- Les **biais nuls** sont un choix courant.
- Adapter l'échelle des poids à la couche et à la fonction d'activation.

Des unités d'une même couche ayant les mêmes poids entrants et les mêmes biais calculent les mêmes activations. Si elles reçoivent aussi les mêmes signaux en aval, leurs gradients restent identiques et elles ne peuvent pas apprendre des caractéristiques distinctes. L'initialisation aléatoire des poids brise cette symétrie. Les biais peuvent commencer à zéro, car les poids aléatoires distinguent les unités.

L'initialisation à zéro est valide pour la régression logistique, qui ne possède pas de couche d'unités cachées dont il faut briser la symétrie. Elle n'exige pas que les classes soient linéairement séparables. Dans un réseau multicouche, fixer tous les poids à zéro peut aussi empêcher des signaux utiles d'atteindre les premières couches. La discussion ultérieure des initialisations Xavier/Glorot et He expliquera l'effet de l'échelle des poids sur l'entraînement.

Propagation avant et perte

- Appliquer la **transformation affine et l'activation** de chaque couche.
- Conserver les **préactivations et activations** nécessaires au passage arrière.
- Pour la classification binaire, la sortie sigmoïde est la probabilité $\hat{p}$.
- Calculer l'**entropie croisée binaire moyenne** sur le lot courant.

Le calcul qui produit les probabilités correspond à `forward()` ou `predict_proba()` dans la présentation 12. Notre méthode d'entraînement conserve aussi les valeurs intermédiaires. La méthode `predict()` applique un seuil aux probabilités pour produire des étiquettes. Cette opération ne fait partie ni du calcul de la perte ni de son passage arrière.

Pour un exemple, on calcule $\ell(y, \hat{p})$. Pour un lot de B exemples, on calcule $J_B = B^{-1} \sum_i \ell(y_i, \hat{p}_i)$. D'autres tâches exigent d'autres activations de sortie et d'autres pertes. Par exemple, une régression peut utiliser une sortie linéaire et l'erreur quadratique moyenne.

Propagation arrière et mise à jour des paramètres

1. Calculer le **signal d'erreur** de la couche de sortie.
2. Remonter les couches cachées : **somme pondérée des signaux en aval**, puis **dérivée de l'activation**.
3. Utiliser les activations et signaux d'erreur de chaque couche pour calculer les **gradients de ses paramètres**.
4. Une fois tous les gradients disponibles, mettre les paramètres à jour :

$$\theta \leftarrow \theta - \alpha \nabla_\theta J_B.$$

Le passage arrière réutilise les valeurs et paramètres du passage avant qui ont produit la perte courante. Modifier une couche ultérieure avant de calculer les gradients des couches précédentes mélangerait les paramètres de réseaux différents. Notre implémentation conserve donc tous les gradients des paramètres avant d'appliquer les mises à jour.

Dans l'exemple à un seul chemin, le signal d'erreur est multiplié par un poids en aval. Dans l'exemple avec embranchement, il reçoit deux contributions pondérées. Le réseau général effectue la même opération par multiplication matricielle. La rétropropagation calcule les dérivées des opérations élémentaires plutôt que de perturber chaque paramètre pour estimer son effet sur la perte.

Implémentation

Rétropropagation : passage avant

- Calculer les activations couche par couche :

  $z^{(l)} = a^{(l-1)} W^{(l)} + b^{(l)}$,

  $a^{(l)} = \varphi(z^{(l)})$.

- Pour la classification binaire, la sortie est une probabilité $\hat{p}$. Calculer la perte d'un exemple $\ell(y, \hat{p})$.

Rétropropagation : passage arrière

- **Couche de sortie**, pour un exemple avec sortie sigmoïde et entropie croisée binaire :

  $$\delta^{(L)} = \hat{p} - y.$$

- **Couches cachées**, dans l'ordre inverse $l = L-1, \ldots, 1$:

$$\delta^{(l)} = (\delta^{(l+1)}(W^{(l+1)})^\top) \odot \varphi'(z^{(l)})$$

Le symbole $\odot$ désigne le produit élément par élément, ou produit de Hadamard.

L'équation de sortie reprend $\delta_2 = \hat{p} - y$ de l'exemple scalaire en utilisant maintenant L pour la couche finale. Elle s'applique à notre classificateur binaire avec une seule sortie sigmoïde et l'entropie croisée binaire. On commence par ce signal, puis on applique la récurrence des couches cachées de $L - 1$ jusqu'à 1. Pour un lot, les signaux de sortie correspondants sont `A_L - yb`. On calcule la moyenne sur les exemples lors de la formation des gradients des paramètres.

- $\delta^{(l)}$ est la ligne de **signaux d'erreur** de la couche l pour un exemple, définie par $\delta^{(l)} = \dfrac{\partial \ell}{\partial z^{(l)}}$, soit la sensibilité de la perte de cet exemple aux préactivations de la couche.
- $\delta^{(l+1)}$ contient les **signaux d'erreur de la couche suivante**, plus proche de la sortie. Ils sont déjà connus grâce à l'étape précédente de rétropropagation.
- $\delta^{(l+1)}(W^{(l+1)})^\top$ propage les signaux de la couche suivante **vers l'arrière à travers les poids** de la couche $l + 1$. Chaque neurone de la couche l reçoit une somme pondérée des signaux en aval, comme a recevait $r\delta_u + s\delta_v$ dans l'exemple avec embranchement. La transposition correspond à notre convention de vecteurs lignes et aux matrices de poids de forme `(units_in, units_out)`.
- $\varphi'(z^{(l)})$ est la **dérivée de la fonction d'activation** à la couche l. Elle module le signal selon la sensibilité de l'activation du neurone aux variations de son entrée.

D'un exemple à un lot

Empiler les activations et signaux d'erreur de chaque exemple comme lignes de $A^{(l)}$ et $\Delta^{(l)}$.

Pour un lot de B exemples :

$$\frac{\partial J_B}{\partial W^{(l)}} = \frac{1}{B}(A^{(l-1)})^\top \Delta^{(l)}, \qquad \frac{\partial J_B}{\partial b^{(l)}} = \frac{1}{B}\sum_{i=1}^{B} \delta_i^{(l)}.$$

```
grad_W = activations_prev.T @ delta / batch_size
grad_b = delta.mean(axis=0)
```

Dans la somme, i indexe les exemples : $\delta_i^{(l)}$ est donc la ligne de signaux de l'exemple i. Ici, $\Delta^{(l)}$ empile les dérivées de la perte propre à chaque exemple, ℓ_i, par rapport à ses préactivations. Ce n'est pas encore la dérivée de la perte moyenne J_B par rapport à toute la matrice $Z^{(l)}$ du lot, qui inclurait déjà le facteur $1/B$. Le tableau `delta` du code

suit cette convention par exemple. La moyenne intervient une seule fois lors du calcul des gradients des paramètres.

Pour une sortie sigmoïde et l'entropie croisée binaire, les signaux de sortie sont `delta = A_L - yb`, les deux tableaux ayant la forme `(B, 1)`. Chaque couche cachée applique `delta = (delta @ W_next.T) * activation_derivative`. Ce calcul additionne les contributions pondérées en aval séparément pour chaque exemple. Il ne calcule pas de moyenne sur les unités.

Si une couche possède n_{l-1} entrées et n_l unités, ses activations ont la forme `(B, n_l)`, ses poids `(n_prev, n_l)` et ses signaux d'erreur `(B, n_l)`. Ainsi, `activations_prev.T @ delta` a exactement la forme de la matrice de poids. La somme des contributions aux biais sur les exemples donne un vecteur de forme `(n_l,)`, comme le vecteur de biais.

SimplerMLP

Unités sigmoïdes, entropie croisée binaire et taux d'apprentissage constant.

In [10]:
```python
import numpy as np

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_prime(z):
    a = sigmoid(z)
    return a * (1.0 - a)

def bce_loss(y_true, y_prob, eps=1e-9):
    """Entropie croisée binaire moyennée sur les exemples."""
    p = np.clip(y_prob, eps, 1 - eps)
    return -np.mean(y_true * np.log(p) + (1 - y_true) * np.log(1 - p))

def xavier_init(rng, fan_in, fan_out):
    std = np.sqrt(2.0 / (fan_in + fan_out))
    return rng.normal(0.0, std, size=(fan_in, fan_out))

class SimplerMLP:
    """MLP sigmoïde pour la classification binaire; taux constant."""

    def __init__(self, layer_sizes, lr=0.1, seed=None):
        self.sizes = list(layer_sizes)
        self.lr = float(lr)
        self.rng = np.random.default_rng(seed)

        self.W = []
        for din, dout in zip(self.sizes[:-1], self.sizes[1:]):
            Wk = xavier_init(self.rng, din, dout)
            self.W.append(Wk)
        self.b = [np.zeros(dout) for dout in self.sizes[1:]]
```

```
    def forward(self, X):
        a = X
        for W, b in zip(self.W, self.b):
            a = sigmoid(a @ W + b)
        return a.ravel()

    def predict_proba(self, X):
        return self.forward(X)

    def predict(self, X, threshold=0.5):
        return (self.predict_proba(X) >= threshold).astype(int)

    def loss(self, X, y):
        p = self.predict_proba(X)
        return bce_loss(y, p)

    def _forward_full(self, X):
        a = X
        activations = [a]
        zs = []
        for W, b in zip(self.W, self.b):
            z = a @ W + b
            a = sigmoid(z)
            zs.append(z)
            activations.append(a)
        return activations, zs

    def train(self, X, y, epochs=30, batch_size=None, verbose=True):
        """X : (N, d), y : (N,); batch_size=None utilise tout le jeu de donr
        N = X.shape[0]
        idx = np.arange(N)
        B = N if batch_size is None else int(batch_size)

        for ep in range(1, epochs + 1):
            self.rng.shuffle(idx)

            base_loss = self.loss(X, y)

            for start in range(0, N, B):
                sl = idx[start:start + B]
                Xb = X[sl]
                yb = y[sl].reshape(-1, 1)
                Bsz = Xb.shape[0]

                activations, zs = self._forward_full(Xb)
                A_L = activations[-1]

                # Chaque ligne est la sensibilité aux préactivations d'un ex
                delta = A_L - yb
                grads_W = [None] * len(self.W)
                grads_b = [None] * len(self.b)

                # Calculer une seule fois la moyenne des gradients sur les e
                grads_W[-1] = activations[-2].T @ delta / Bsz
                grads_b[-1] = delta.mean(axis=0)
```

```
                for k in range(len(self.W) - 2, -1, -1):
                    z = zs[k]
                    d_act = sigmoid_prime(z)
                    delta = (delta @ self.W[k + 1].T) * d_act
                    grads_W[k] = activations[k].T @ delta / Bsz
                    grads_b[k] = delta.mean(axis=0)

                # Tous les gradients utilisent les poids du même passage ava
                for k in range(len(self.W)):
                    self.W[k] -= self.lr * grads_W[k]
                    self.b[k] -= self.lr * grads_b[k]

            new_loss = self.loss(X, y)
            if verbose:
                print(f"Époque {ep:3d} | perte {base_loss:.5f} -> {new_loss:
```

L'implémentation complète reprend la structure de `NaiveMLP` de la présentation 12, mais remplace les essais de modification des paramètres par les gradients calculés par rétropropagation. Toutes les unités cachées et de sortie utilisent la sigmoïde. La sortie fournit une probabilité par exemple.

La classe ne comporte ni régularisation ni calendrier de taux d'apprentissage. La présentation 14 étend cette même structure sous le nom `SimpleMLP`. Les deux classes utilisent les mêmes méthodes de base, la même organisation des paramètres et des listes de valeurs conservées, ainsi que le même calcul arrière. `SimpleMLP` ajoute des options de configuration. Les carnets sont autonomes : aucun héritage ni importation d'un carnet dans l'autre n'est nécessaire.

Activation et dérivée

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_prime(z):
    a = sigmoid(z)
    return a * (1.0 - a)
```

`sigmoid_prime` reçoit la préactivation z, et non l'activation a. Elle calcule $a = \sigma(z)$ et renvoie $a(1 - a)$. Il s'agit d'une dérivée locale, pas du gradient complet d'un paramètre.

`SimplerMLP` appelle directement `sigmoid` et `sigmoid_prime`, puisque chaque unité utilise la même activation. La présentation 14 introduit des méthodes de sélection de l'activation lorsque `SimpleMLP` ajoute un choix pour les unités cachées.

Perte

```
def bce_loss(y_true, y_prob, eps=1e-9):
    """Entropie croisée binaire moyennée sur les exemples."""
    p = np.clip(y_prob, eps, 1 - eps)
```

```
    return -np.mean(y_true * np.log(p) + (1 - y_true) * np.log(1 -
p))
```

`forward` et `predict_proba` renvoient des probabilités. Seule `predict` leur applique un seuil pour produire des étiquettes de classe. La perte est l'entropie croisée binaire moyenne, sans terme de pénalité. Le bornage évite de prendre le logarithme d'une probabilité en virgule flottante égale à zéro ou à un. Les bibliothèques utilisées en pratique calculent souvent une perte combinée à partir des logits pour améliorer la stabilité numérique.

Initialisation des paramètres

```
def xavier_init(rng, fan_in, fan_out):
    std = np.sqrt(2.0 / (fan_in + fan_out))
    return rng.normal(0.0, std, size=(fan_in, fan_out))
```

Les poids sont tirés aléatoirement et leur échelle dépend des largeurs des couches. Les biais commencent à zéro. Il s'agit de l'initialisation Xavier/Glorot utilisée pour les unités sigmoïdes. La motivation mathématique et les autres choix d'initialisation sont développés dans la présentation 14. Le générateur créé à partir de `seed` contrôle l'initialisation et l'ordre des exemples pendant l'entraînement.

Définition de la classe et constructeur

```
class SimplerMLP:

    def __init__(self, layer_sizes, lr=0.1, seed=None):
        self.sizes = list(layer_sizes)
        self.lr = float(lr)
        self.rng = np.random.default_rng(seed)

        self.W = []
        for din, dout in zip(self.sizes[:-1], self.sizes[1:]):
            Wk = xavier_init(self.rng, din, dout)
            self.W.append(Wk)
        self.b = [np.zeros(dout) for dout in self.sizes[1:]]
```

Le constructeur accepte uniquement l'architecture, le taux d'apprentissage et la graine aléatoire. Les matrices de poids ont la forme `(units_in, units_out)` et les biais `(units_out,)`, conformément au calcul `a @ W + b`.

Forward (public)

```
    def forward(self, X):
        a = X
        for W, b in zip(self.W, self.b):
            a = sigmoid(a @ W + b)
        return a.ravel()
```

`forward()` renvoie des probabilités de forme `(N,)`. `predict_proba()` donne accès au même calcul, tandis que `predict()` applique un seuil. Chaque couche utilise la sigmoïde : la boucle n'a donc besoin ni d'un compteur de couches ni d'un cas particulier pour la sortie. `SimpleMLP` conserve ce calcul couche par couche, mais ajoute la sélection de l'activation dans la présentation 14.

Forward (privé)

```
    def _forward_full(self, X):
        a = X
        activations = [a]
        zs = []
        for W, b in zip(self.W, self.b):
            z = a @ W + b
            a = sigmoid(z)
            zs.append(z)
            activations.append(a)
        return activations, zs
```

La méthode `_forward_full` conserve les préactivations et les activations pour la rétropropagation. La couche mathématique l correspond à `activations[l]` et `zs[l - 1]`. L'entrée `activations[0]` contient la matrice X. Les deux classes conservent ces tableaux dans le même ordre.

Entraînement

```
    def train(self, X, y, epochs=30, batch_size=None,
verbose=True):
        """X : (N, d), y : (N,); batch_size=None utilise tout le
jeu de données."""
        N = X.shape[0]
        idx = np.arange(N)
        B = N if batch_size is None else int(batch_size)

        for ep in range(1, epochs + 1):
            self.rng.shuffle(idx)

            base_loss = self.loss(X, y)

            for start in range(0, N, B):
                sl = idx[start:start + B]
                Xb = X[sl]
                yb = y[sl].reshape(-1, 1)
                Bsz = Xb.shape[0]

                activations, zs = self._forward_full(Xb)
                A_L = activations[-1]

                # Chaque ligne est la sensibilité aux
```

```
préactivations d'un exemple.
                delta = A_L - yb
                grads_W = [None] * len(self.W)
                grads_b = [None] * len(self.b)

                # Calculer une seule fois la moyenne des gradients
sur les exemples.
                grads_W[-1] = activations[-2].T @ delta / Bsz
                grads_b[-1] = delta.mean(axis=0)

                for k in range(len(self.W) - 2, -1, -1):
                    z = zs[k]
                    d_act = sigmoid_prime(z)
                    delta = (delta @ self.W[k + 1].T) * d_act
                    grads_W[k] = activations[k].T @ delta / Bsz
                    grads_b[k] = delta.mean(axis=0)

                # Tous les gradients utilisent les poids du même
passage avant.
                for k in range(len(self.W)):
                    self.W[k] -= self.lr * grads_W[k]
                    self.b[k] -= self.lr * grads_b[k]

            new_loss = self.loss(X, y)
            if verbose:
                print(f"Époque {ep:3d} | perte {base_loss:.5f} ->
{new_loss:.5f} | lr={self.lr:.4f}")
```

Le signal de sortie `A_L - yb` est la dérivée sigmoïde/BCE déjà simplifiée. Il ne faut pas le multiplier à nouveau par la dérivée de la sigmoïde. Les signaux cachés exigent une multiplication par les poids en aval et par la dérivée locale de la sigmoïde.

La boucle arrière utilise un indice Python `k` : `W[k]` appartient à la couche mathématique $k+1$. À cette étape, `delta` contient les signaux de la couche suivante. Le calcul `delta @ W[k + 1].T` additionne leurs contributions à la couche courante. Le gradient du biais est la moyenne de ces signaux sur les exemples, tandis que celui des poids inclut aussi les activations précédentes.

Tous les gradients sont calculés avant toute mise à jour des paramètres. Le taux d'apprentissage reste constant pendant l'entraînement.

Cette implémentation suppose des exemples d'entraînement indépendants et mélange leurs indices au début de chaque époque. Les mêmes indices sélectionnent les lignes de `X` et de `y`, de sorte que les caractéristiques restent associées à leurs cibles. Le mélange change les exemples réunis dans un mini-lot et évite de présenter toujours la même séquence de lots.

Si l'entraînement dépend de l'ordre chronologique ou séquentiel, l'implémentation doit être modifiée pour le préserver. Par exemple, un modèle séquentiel avec état peut transmettre de l'information d'un lot au suivant. Les données temporelles n'interdisent

pas toujours le mélange : des fenêtres temporelles traitées indépendamment peuvent être mélangées après une séparation chronologique entre entraînement, validation et test, à condition de conserver l'ordre à l'intérieur de chaque fenêtre. Toute autre procédure d'entraînement qui dépend volontairement de l'ordre des exemples exige aussi une autre politique. Le tutoriel TensorFlow sur les séries temporelles illustre le mélange de fenêtres d'entraînement, tandis que les consignes Keras pour les RNN avec état exigent des lots ordonnés.

Évaluation

In [11]:
```
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

X, y = make_circles(n_samples=200, factor=0.5, noise=0.08, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0, stratify=y)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SimplerMLP([2, 4, 4, 1], lr=0.3, seed=42)
model.train(X_train, y_train, epochs=2000, batch_size=32, verbose=False)

print("Exactitude (entraînement) :", accuracy_score(y_train, model.predict(X
print("Exactitude (test) :", accuracy_score(y_test, model.predict(X_test)))
```

```
Exactitude (entraînement) : 1.0
Exactitude (test) : 1.0
```

Le jeu de données circulaire et sa séparation entraînement/test sont les mêmes que dans l'exemple ultérieur avec `SimpleMLP`. La standardisation est ajustée uniquement sur les données d'entraînement. Ce réseau sigmoïde peu profond utilise un taux d'apprentissage constant et davantage d'époques. Les réglages illustrent un exemple complet qui fonctionne, plutôt qu'une comparaison de modèles optimisés. L'exactitude de test est rapportée après l'entraînement et ne sert pas à choisir les réglages.

Différentiation automatique

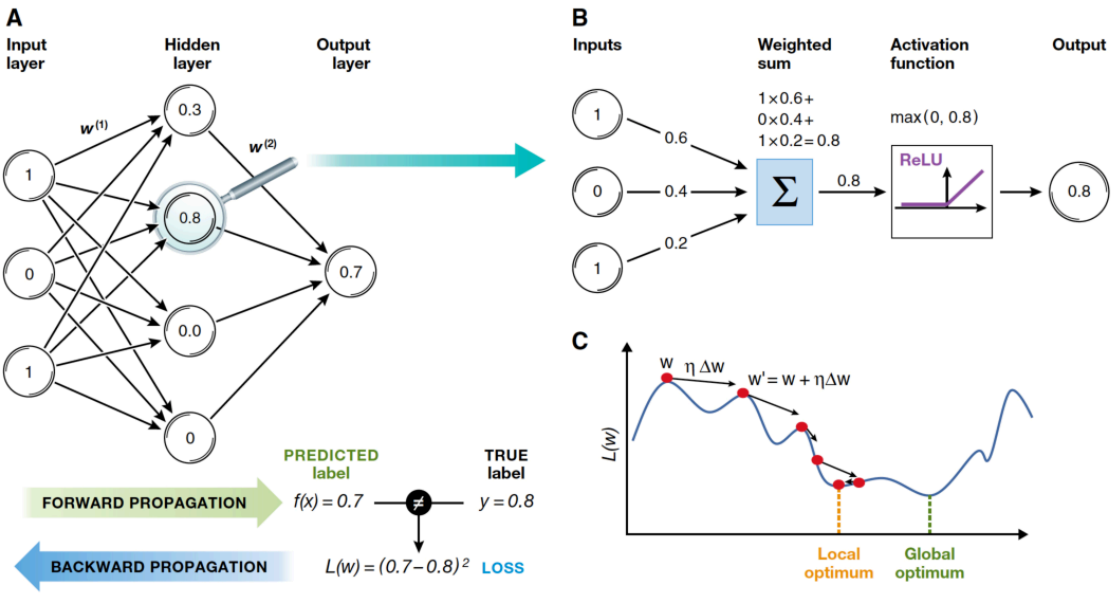
La **différentiation automatique (autodiff)** applique systématiquement la règle de la chaîne pour calculer les dérivées exactes de fonctions exprimées sous forme de programmes. Elle propage les dérivées à travers les opérations élémentaires, soit vers l'avant, des entrées aux sorties, soit vers l'arrière, des sorties aux entrées. Elle permet ainsi de calculer efficacement les gradients utilisés en optimisation et en apprentissage.

Baydin et al. (2017)

Ici, « exact » distingue l'utilisation des règles de dérivation locales des approximations par différences finies. Les valeurs calculées restent soumises aux erreurs d'arrondi en virgule flottante. La différentiation automatique n'a pas besoin de développer une expression symbolique complète de la perte. SymPy permet d'obtenir ou de vérifier les formules locales, tandis qu'un moteur de différentiation automatique applique ces formules aux valeurs et aux opérations rencontrées pendant l'exécution.

Épilogue

Résumé



Attribution : Angermueller et al. (2016)

Résumé

Concept	Rôle
Fonctions d'activation	Introduisent la non-linéarité (sigmoïde dans `SimplerMLP`).
Fonction de perte	Mesure l'erreur de prédiction, par exemple avec l'entropie croisée binaire.
Taux d'apprentissage (α)	Contrôle la taille des mises à jour des paramètres.
Descente de gradient	Méthode d'optimisation pour ajuster les poids.
Règle de la chaîne	Mécanisme de propagation des dérivées vers l'arrière.
Différentiation automatique	Automatise le calcul des dérivées. La rétropropagation utilise le mode inverse.

De la rétropropagation à l'implémentation

- Conserver les activations et préactivations du passage avant.
- Calculer les signaux d'erreur vers l'arrière et additionner les contributions aux embranchements.
- Calculer la moyenne des gradients des paramètres sur les exemples, puis mettre tous les paramètres à jour.
- `SimplerMLP` implémente ce processus avec des unités sigmoïdes et un taux d'apprentissage constant.

3Blue1Brown

Une série de vidéos animées qui développe l'**intuition** derrière l'**algorithme de rétropropagation**.

- Réseaux de neurones (liste de lecture)
  - Que fait réellement la rétropropagation? (12 min 47 s)
  - Les calculs de la rétropropagation (10 min 18 s)

Préalable : La descente de gradient : comment les réseaux de neurones apprennent-ils? (20 min 33 s)

StatQuest

- Réseaux de neurones, partie 2 : les idées principales de la rétropropagation (17 min 34 s)
- Détails de la rétropropagation, partie 1 : optimiser trois paramètres simultanément (18 min 32 s)
- Détails de la rétropropagation, partie 2 : explorer la règle de la chaîne (13 min 9 s)

Préalables : La règle de la chaîne (18 min 24 s) et La descente de gradient, étape par étape (23 min 54 s)

Herman Kamper

Une série de **vidéos** particulièrement approfondie sur l'**algorithme de rétropropagation**.

- Introduction aux réseaux de neurones (liste de lecture)
  - Rétropropagation sans embranchements (31 min 1 s)
  - Rétropropagation pour un réseau à propagation avant multicouche (4 min 2 s)

- Graphes de calcul et différentiation automatique pour les réseaux de neurones (6 min 56 s)
- Dérivées courantes pour les réseaux de neurones (7 min 18 s)
- Une notation générale pour les dérivées dans les réseaux de neurones (7 min 56 s)
- Embranchements dans les réseaux de neurones (13 min 46 s)
- Rétropropagation générale avec embranchements (3 min 42 s)

Livre gratuit avec implémentation

Dans son livre *Neural Networks and Deep Learning*, Michael Nielsen fournit une implémentation Python complète d'un réseau de neurones.

Prochaine présentation

- Propagation des gradients, fonctions d'activation et initialisation.
- Extension de `SimplerMLP` en `SimpleMLP` avec choix d'activations, régularisation L2 et décroissance du taux d'apprentissage.
- Softmax, entropie croisée et régularisation.

Références

Angermueller, Christof, Tanel Pärnamaa, Leopold Parts, et Oliver Stegle. 2016. « Deep learning for computational biology ». *Mol Syst Biol* 12 (7): 878. https://doi.org/10.15252/msb.20156651.

Baydin, Atılım Günes, Barak A. Pearlmutter, Alexey Andreyevich Radul, et Jeffrey Mark Siskind. 2017. « Automatic differentiation in machine learning: a survey ». *J. Mach. Learn. Res.* 18 (1): 5595-637.

Bicknell, Klinton, Claire Brust, et Burr Settles. 2023. « How Duolingo's AI Learns what you Need to Learn: The language-learning app tries to emulate a great human tutor ». *IEEE Spectrum* 60 (3): 28-33. https://doi.org/10.1109/MSPEC.2023.10061631.

Rumelhart, David E., Geoffrey E. Hinton, et Ronald J. Williams. 1986. « Learning representations by back-propagating errors ». *Nature* 323 (6088): 533-36. https://doi.org/10.1038/323533a0.

Russell, Stuart, et Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4e éd. Pearson. http://aima.cs.berkeley.edu/.

Annexe : Dérivées principales (SymPy)

Dérivées principales avec SymPy

In [12]:
```python
import sympy as sp
from IPython.display import Math, display

def sigmoid(z):
    return 1 / (1 + sp.exp(-z))

x, y = sp.symbols("x y", real=True)
w1, b1, w2, b2 = sp.symbols("w_1 b_1 w_2 b_2", real=True)

# Expressions imbriquées pour que la perte conserve sa dépendance à chaque p

z1 = w1 * x + b1
a1 = sigmoid(z1)

z2 = w2 * a1 + b2
p_hat = sigmoid(z2)

loss = -(y * sp.log(p_hat) + (1 - y) * sp.log(1 - p_hat))

gradients = {parameter: sp.diff(loss, parameter) for parameter in (w2, b2, w

# Réécriture pour améliorer la lisibilité

a1_symbol, p_symbol = sp.symbols("a_1 p_hat", real=True)

rewrites = [
  (sp.exp(-z2), (1 - p_symbol) / p_symbol),
  (sp.exp(-z1), (1 - a1_symbol) / a1_symbol),
]

for parameter, gradient in gradients.items():
    named_gradient = sp.factor(gradient.subs(rewrites))
    lhs = rf"\frac{{\partial \ell}}{{\partial {sp.latex(parameter)}}}"
    rhs = sp.latex(named_gradient, symbol_names={p_symbol: r"\hat p"})
    display(Math(f"{lhs} = {rhs}"))
```

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

École de génie électrique et d'**informatique**

Université d'Ottawa